Linked List

Disadvantages of Array

1. Elements of arrays are always stored in contiguous memory
    a. Inserting or deleting an element in an array may require all of its elements to be shifted
2. The size of array is always fixed
    a. You cannot add a new element beyond the end of the array
    b. Memory for the entire array is always reserved even though you use only part of the array
    c. You must guess the expected maximum size of the list in advance

Linked List

A **linked list** is a series of data items with each item containing a pointer giving a location of the next item in the list. A **linked list** is a collection of nodes, where each node consists of two parts

- **info**: the actual element to be stored in the list.
- **link**: one or two links that points to next and previous node in the list. also known as pointer.

The entire link list is accessed by an external pointer (say list) that points to the first node in the linked list.

Basic Operations on linked list

- Insertion

This operation creates a new node and inserts it into the list. A new node may be inserted

- at the beginning of the list (insertHead)
- at the end of the list (insertTail)
- at the specified position (insertAfter, insertBefore,insertAtNthPos)
- if the list is empty, the node to be inserted will be the first node

- Deletion

This operation deletes a node (i.e. an item) from the linked list. A node may be deleted from the

- beginning of the list (deleteHead)
- end of the list (deleteTail)
- specified position (deleteAfter, deleteAtNthPos etc)

- Traversing

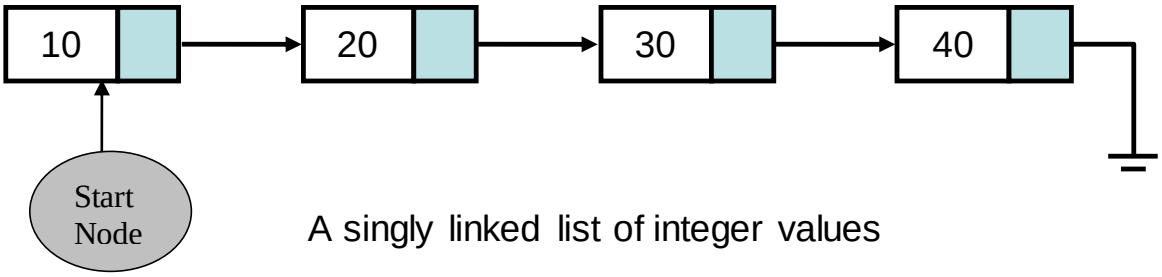
The process is visiting all the nodes of the linked list from one end to the other is termed as traversing the list.

- Searching

This operation checks if a particular item is present in the list.

-Display

This operation is used to print the content of the list.



A singly linked list of integer values

i. The next address field of the **last node** in the list contains a special value, known as **null**, which is not a valid address. This null pointer is used to signal the end of a list.
ii. The list with **no nodes** on it is called the **empty list** or the null list.
iii. The nodes in a linked list are not stored contiguously in the computer's memory.
iv. We don't have to **shift** any element while **inserting/deleting** in the list
v. Memory for each node can be allocated dynamically whenever the need arises
vi. The no of items in the list is only limited by the computer's memory

Creation of Linked List

Linked list can be created as:

```
struct list
{
  int item;
   struct list *next;
};
typedef struct list node;
node *ptr, *start;
ptr = (node *) malloc (sizeof(node));
```

The struct declaration merely describes the format of the nodes and does not allocate storage. Here, the type of the information to be stored is 'int'. next is a pointer to the parent structure type. This type of structure is called self-referential structures. Storage space for a node is created only when the function malloc() is called in the statement

ptr = (node *)malloc(sizeof(node)); - (1)
start = ptr; - (2)

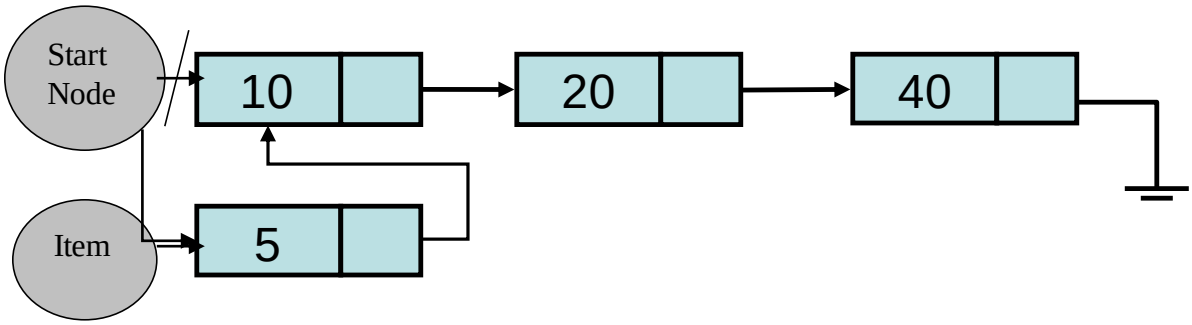
This statement (1) obtains a piece of memory that is sufficient to store a node and assigns (2) its address to the pointer variable **start**. This **start** pointer indicates the beginning of the linked list.

Insertion

To insert a node in a list,

- allocate required memory to a node
- assign the data in its information field
- adjust the pointers to link the node at the required place

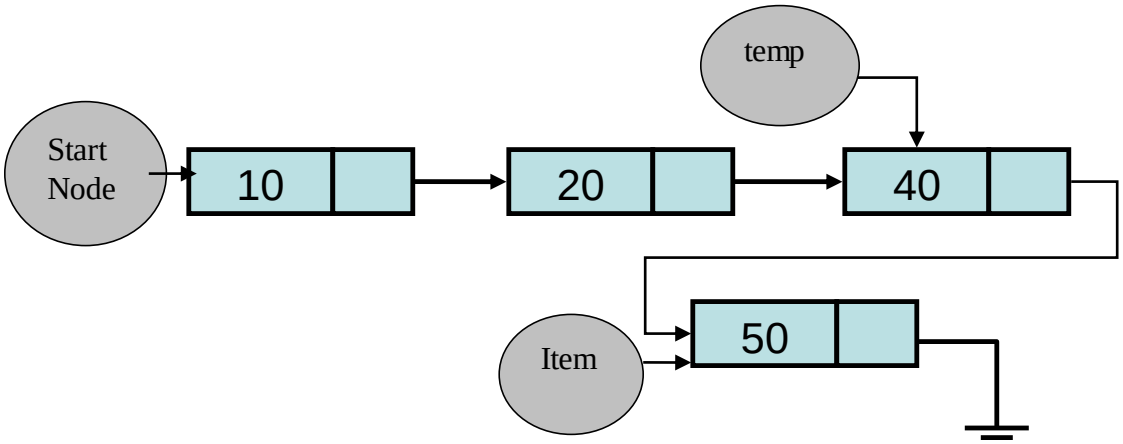
1. Algorithm for inserting a new node at the **beginning** of the list
   i. Allocate memory for the new node
   ii. Assign value
   iii. If the list is currently empty, assign NULL to the link field of the new node. Otherwise, make the link field of the new node to point to the starting node of the linked list.
   iv. Then, set the external pointer (which is pointing to the starting node) to point to the new node.



C Code

```
Void f_insert()
{
        node *item;
        item=(node *)malloc(sizeof(node));
        printf("Enter data to insert");
        scanf("%d",&item->info);
        if(start==NULL)
        {
                item->next=NULL;
                start=item;
        }
        else
        {
                item->next=start;
                start=item;
        }
}
```

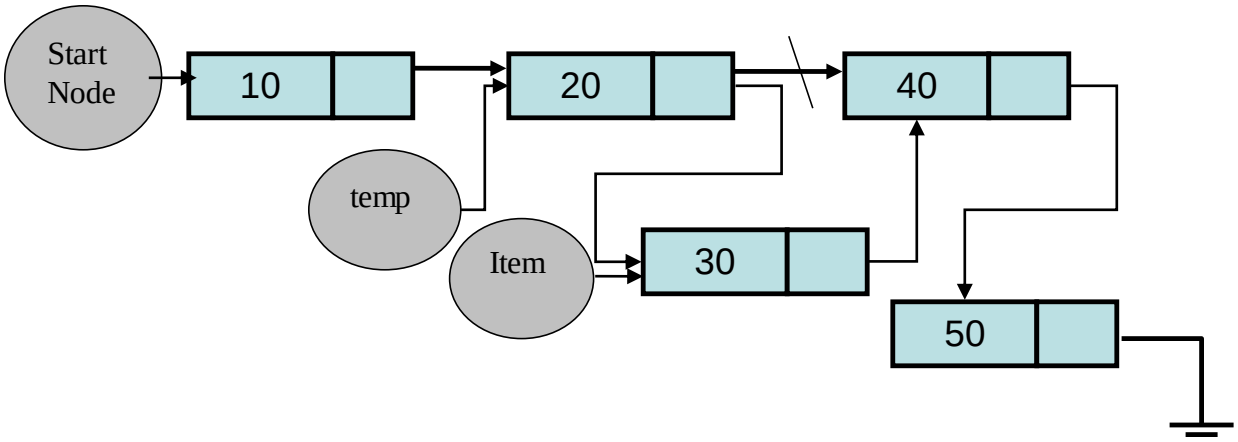
2. Algorithm for inserting a new node at the **end** of the list
   i. Allocate memory for the new node
   ii. Assign value
   iii. Assign NULL to the link field of the new node.
   iv. Traverse the list until last node is reached. Then, insert the new node after the last node by storing the address of the newnode (item) to link field of the last node.



C Code

```
void e_insert()
        {
                node *item,*temp;
                item=(node *)malloc(sizeof(node));
                printf("\nEnter data to insert:");
                scanf("%d",&item->info);
                item->next=NULL;
                if(start==NULL)
                start=item;
                else
                {
                        temp=start;
                        while(temp->next!=NULL)
                                temp=temp->next;
                        temp->next=item;
                }
        }
```

3. Algorithm for inserting a new node at specific position
   i. Allocate memory for the new node
   ii. Assign value
   iii. Assign next of the given node to the link field of the new node.
   iv. Assign newnode (item) to the link field of the given node



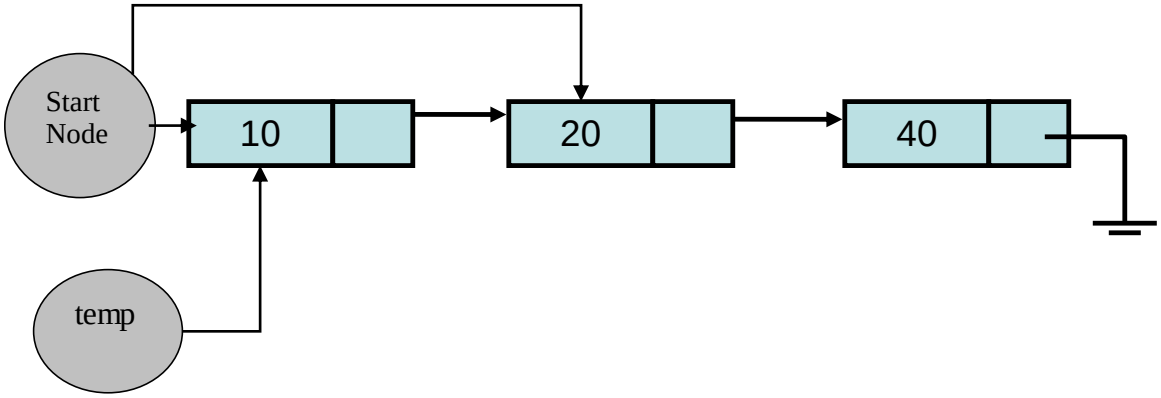
C Code

```
Void n_insert()
{
        int n,count=1;
        node *ptr,*temp;
        item=(node *)malloc(sizeof(node));
        printf("\nEnter the position");
        scanf("%d",&n);
        printf("enter the data to insert");
        scanf("%d",&item->info);
        temp=start;
        while(count<n)
        {
                temp=temp->next;
                count++;
        }
        item->next=temp->next;
        temp->next=temp;
}
```

Deletions

To delete a node from a list
– Find out the node to be deleted.
– Adjust the pointers
– Free the node to be deleted

Algorithm for deleting the **first** node
1. If the list is empty, it is underflow.
2. Else,
   – move the start pointer to the second node
   – free the first node



C Code

```
void f_del()
{
        node *ptr;
        if(start==NULL)
        {
                printf("\n Empty list");
                getch();
        }
        else if(start->next==NULL)
        {
                printf("\nDeleted element is %d",start>info);
                getch();
                free(start);
                start=NULL;
        }
        else
        {
                ptr=start;
                start=start->next;
                printf("\n Deleted element is%d",ptr->info);
                getch();
                free(ptr);
        }
}
```

Algorithm for deleting the **last** node

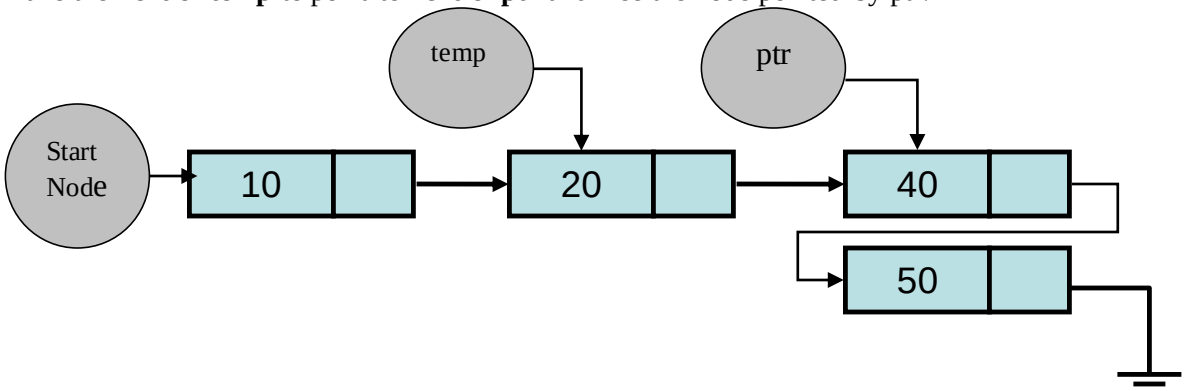
1. If the list is empty, it is underflow
2. Else, traverse until the last node is encountered.
   - Also keep track of the node, which is before the current node.
3. make the second last node (i.e. temp1) to point to NULL and free the last node(i.e. current)

C Code

```
void e_del()
{
        node *ptr,*loc;
        if(start==NULL)
        {
                printf("\nEmpty list");
                getch();
        }
        else if(start->next==NULL)
        {
                printf("\nDeleted item is %d",start->info);
                free(start);
                start=NULL;
        }
        else
        {
                ptr=start;
                while(ptr->next!=NULL)
                {
                        loc=ptr;
                        ptr=ptr->next;
                }
                loc->next=NULL;
                printf("\nDeleted element is %d",ptr->info);
                getch();
                free(ptr);
        }
}
```

Algorithm for deleting a node at the **specified position (n)**

1. If the list is empty, it is underflow
2. Else, traverse until the specified node is encountered (ptr)
   - Also keep track of the node, which is before the specified node.
3. make the **next of temp** to point to **next of ptr** and free the node pointed by ptr.



C Code

```
void n_del()
{
    node *temp,*ptr
    int n,count=1;
    if(start==NULL)
    {
        printf("\nEmpty list");
        getch();
    }
    else
    {
        printf("\n Enter a position");
        scanf("%d",&n);
        ptr=start;
        while(count<n)
        {
                count++;
                temp=ptr;
                ptr=ptr->next;
        }
        temp->next=ptr->next;
        printf("\nDeleted item is %d",ptr->info);
        getch();
        free(ptr);
    }
}
```

Traversing a list and displaying information stored

As has been stated, list traversal is the process of visiting all the nodes in the list. Traversal is generally needed to print the items stores in each node of the list. The following function, traverse the given list and prints the item stored in each of them.

Algorithm for **traversing** and **displaying**

1. If the list is empty, print no element exist.
2. Else, traverse until the temp is not NULL (temp!=NULL)
   -While traversing print the info part each time.

C Code

```
void display()
{
        node *temp;
        printf("\nElements of linked list are");
        temp=start;

        while(temp!=NULL)
        {
                printf("%d\t",temp->info);
                temp=temp->next;
        }
```

```
        getch();
}
```

Searching for an item in a list

In order to find an item in a given list, we visit the nodes one after another until we find the item.

1. Start with the first node in the list, as the temp node and keep track of count .
2. Check the info at the current node with the search item
   If the items match, print the value with its position (counter).
   Otherwise, move to the next node and repeat the process
3. If no match is found till we reach the end of the list, print search unsuccessful.

C Code

```
void search()
{
        node *temp;
        int num,count=0;
        printf("\n Enter a number to search");
        scanf("%d",&num);
        temp=start;
        do
        {
                count++;
                if(temp->info==num)
                {
                        printf("\nThis number is at %d position",count);
                        getch();
                        temp=NULL;
                        return;
                }
                else
                        temp=temp->next;
        }
        while(temp!=NULL);
        printf("\nThis number doesnot exist in list");
        getch();
}
```

Destroying a List

When the list is not needed any more, we delete all the nodes of the list by releasing the memory occupied by them.
For this, delete the nodes one by one from the beginning till the last node.

C Code

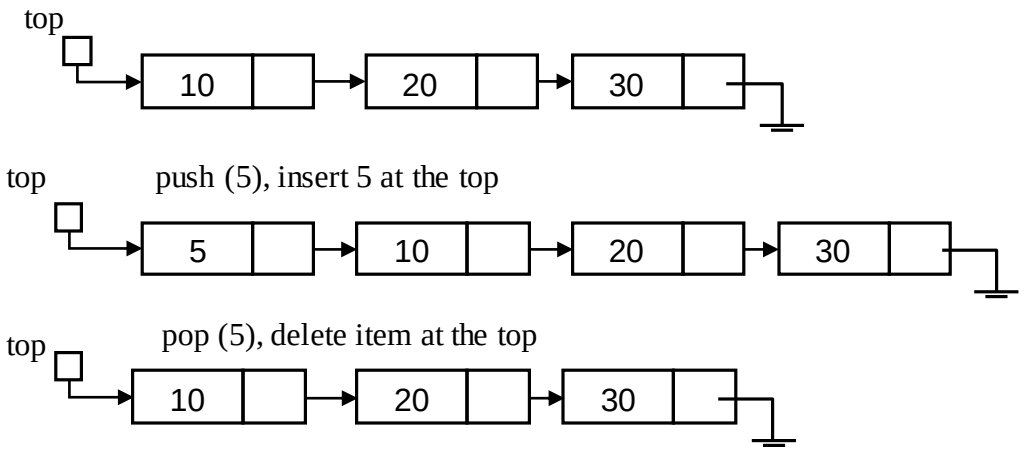
```
void destroy()
{
```

```
        struct node *temp;
        temp = NULL;
        while (start != NULL)
        {
                temp = start;              // Store current node, before deleting
                 start = start->next;      // Move to next node
                free(temp);               // Release memory occupied by temp
        }
}
```

Implementation of Stack using Linked List

As has already been studied, stack is a collection of items in which addition of new items and deletion of existing items are done from only one end known as top of stack. Since, linked list is also a list of items, concepts of stack can be implemented using linked lists instead of an array.

i. The two operations that we apply on a stack are push (inserting item on the top) and pop (deleting item at the top).
ii. The **push** operation is similar to **adding a new node at the beginning of the list** (insertHead()).
iii. A stack items can be accessed only through its top, and a list items can be accessed from the pointer to the first node.
iv. Similarly, **removing the first node** from a linked list (deleteHead()) is similar to **popping** an item from a stack. In both cases, the only immediately accessible item of a collection is removed from that collection, and the next item becomes immediately accessible one.
v. The first node of the list is the top of the stack



Implementing Queue using Linked List

A queue is a special type of list that allows insertion at one end, called the front of queue, and deletion at other end, called the rear of queue.

i. To implement a queue in a linked list, we make the first node of the list as the front and the last node as the rear. So, we will use two pointer variables – front and rear. 'front' will point to the beginning of the list (like start) while 'rear' will point to the last node of the list .
ii. Initially, when the queue is empty, both rear and front will be NULL.
iii. To **enqueue** a new item, we insert it after the rear node and update rear to point to new node. This function will work similarly as insertTail(), because both are the process to add new item after the last node. In the case of Queue, we insert a new item after the node pointed by 'rear'.
iv. To **dequeue** an item, we remove the front node and update front to point to the next node. This function will work similarly as deleteHead() of general list or pop() operation of stack.

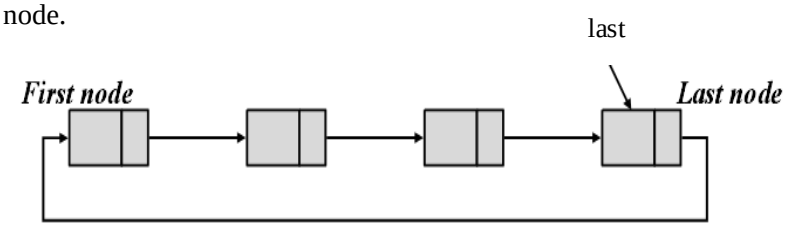
Limitations of Linked Lists

i. Whenever we deal with a fixed length list, it would be better to use an array rather than a linked list.
ii. We must also note that a linked list will use more storage than an array with the same number of items. This is because each item has an additional link field.
iii. To visit a particular list we have to start right from the starting node.

Circular Linked List

This problem can be solved by using a circular linked list. In such a list it is possible to reach any other point in the list. A circular linked list is a list, where the next link of the last node contains a pointer to the first node rather than a null pointer. If we begin at a given node and traverse the entire list, we ultimately end up at the starting point.

In a circular list, we don't have a first or a last node. But, following a convention, we assume that the external pointer points to a last node. The node following this node will be called the first node.



This convention allows access to the last and the first node (by referencing last->next)
Also, this provides the advantage of being able to add or remove an element conveniently from either the front or the rear of the list where as in single linked list, to add a node at the end we have to start right from the beginning to reach the last node.
For an empty circular list is, last = NULL

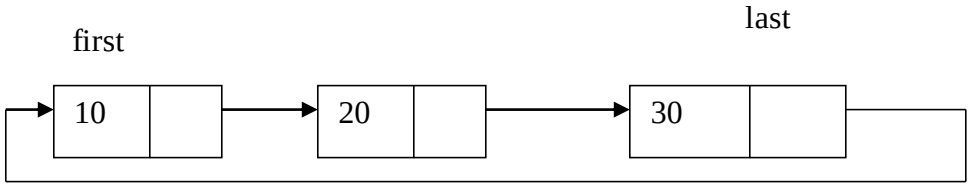


Fig: Circular Linked list

Primitive operations in Circular List

Most of the operations on circular lists are similar to singly linked lists. However, care has to be taken for non-empty lists, as there is no null pointer to signify the end of the list.

We declare the structure for the circular linked list in the same way as we declare it for the linear linked lists.

Struct list

```
{
        int info;
        struct linked_list *next;
}
typedef struct list node;
node *start=NULL;
node *last=NULL;
```

Algorithm for **inserting** a node **at the beginning**

```
1.      ptr=(node *)malloc(sizeof(node))
2.      set ptr->info=item
3.      if start==NULL  then
                set ptr->next=ptr
                set start=ptr
                set last=ptr
        else follow step 4 to  6
4.              set ptr->next=start
5.              set start=ptr
6.              set last->next=ptr
```

Algorithm for **inserting** a new node **at the end**

```
1.      ptr=(node *)malloc(sizeof(node))
2.      set ptr->info=item
3.      if start==NULL
                set ptr->next=ptr
                set start=ptr
                set last=ptr
        else follow step 4 to 6
4.      set last->next=ptr
5.      set last=ptr
6.      set last->next=start
```

Algorithm for **deleting** the node from the **beginning**

```
1.      if start==NULL
                print "circular list is empty"
2.      else follow step 2
                if start->next=start
                        print "Deleted element is start->info"
                        free(start)
                        start=NULL
                        last=NULL
                else follow step 3 to 7
3.                      set ptr = start
4.                      set start = start->next
5.                      print "deleted element is ptr->info"
6.                      set last->next=start
```

7. free(ptr)

Algorithm for **deleting** the node from the **end**

1. if start==NULL
   print "circular list is empty"
   else follow step 2
2. if start->next=start
   print "Delement element is start->info"
   free(start)
   start=NULL
   last=NULL
   else follow step 3 to 10
3. set ptr=start
4. Repeat step 5 and 6 until ptr->next=start
5. set temp=ptr
6. set ptr=ptr->next
7. print "the deleted item is ptr->info"
8. set temp->next=ptr->next
9. set last=temp
10. free(ptr)

Doubly Linked List

In both, singly list and a circular list, list can be traversed only in forward direction. But in doubly list, we can traverse the list in both directions. This has been made possible by defining two address fields, one of them points to the previous(left) node while the other one points to the next(right) node.

The external pointer will point to the first node, as in singly list.
While performing the basic operations, we need to update both(left and right) address fields.

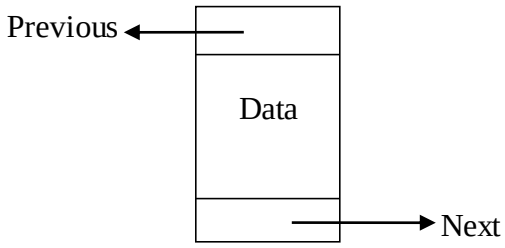


Fig: A node representation with doubly linked list

We declare the structure for the doubly linked as:

```
struct list
{
        int info;
        struct list *prev;
        struct list *next;
};
typedef struct list node;
node *start=NULL;
```

Algorithm for **inserting** a node **at the beginning**

1. ptr=(node *)malloc(sizeof(node))
2. set ptr->info=item
3. if(start==NULL) then
   set ptr->prev = ptr->next = NULL
   set start = ptr
   else follow step 4 to 7
4. set ptr->prev=NULL
5. set ptr->next=start
6. set start->prev=ptr
7. set start=ptr

Algorithm for **inserting** a node **at the end**

1. ptr=(node *) malloc (sizeof(node))
2. ptr->info=item
3. if(start==NULL) then
   set ptr->prev = ptr->next = NULL
   set start = ptr
   else follow step 4 to 7
4. set ptr->next=NULL
5. ptr->prev=last
6. last->next=ptr
7. last=ptr

Trees

Until now we have studied linear data structures. But there are many applications where only the linear data structures are not sufficient. For such situations, we use non-linear data structures like trees and graphs.
In computer science, a tree is an abstract model of a hierarchical structure.

- It is used to represent a hierarchical relationship existing among several data items.
- A tree consists of nodes with a parent-child relationship.
- It is a non-linear data structure.
- Searching as fast as in ordered array.
- Insertion and deletion as fast as in linked list.

Definition of a Tree
It is a finite set of one or more data items (nodes) such that

- There is a special data item called the root of the tree.
- Its remaining data items are partitioned into number of disjoint subsets, each of which is itself a tree again, called subtrees.

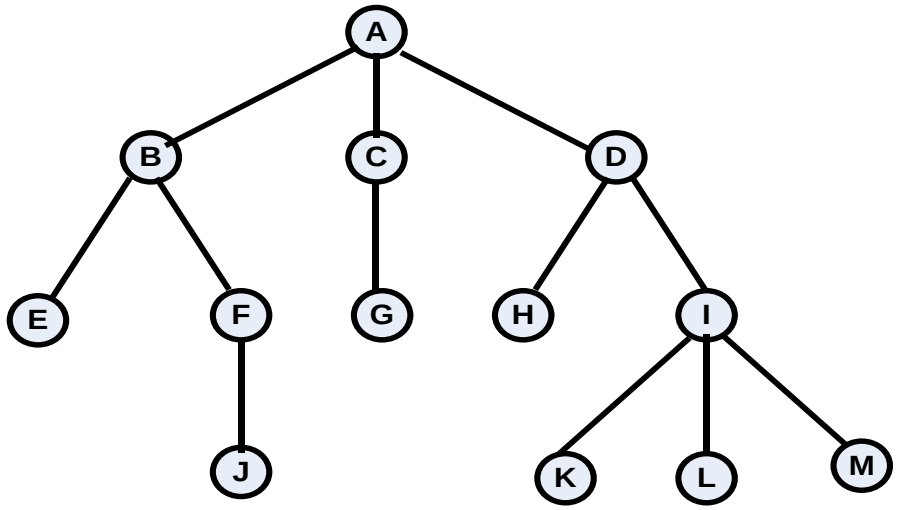


fig: a tree structure

Basic Terminologies in Tree

- **Root: Root** is a special data item, which is the first in the hierarchy. In the above tree,' **A**' is the root item.

- **Node:** Each item in a tree is called a **node**. ***A, B, F, G*** etc. are examples of node.

– **Parent:** A node that points to (one or more) other nodes is the **parent** of those nodes while the nodes pointed to are the **children.** B is parent of E, F and E,F are children of B.

– **Leaf (External Nodes):** Nodes with no children are **leaf** nodes or **external** nodes or **terminal** nodes. In the above tree E, J, G, H, K, L, and M are terminal nodes

– **Internal Nodes:** Nodes with children are **internal nodes** or **non-terminal nodes**. Note that the root node is also an internal node. Node such as **B, C, D**, are non-terminal nodes

– **Siblings (Brother):** Nodes that have the same parent are **siblings**.
In the above tree:
  - **E** and **F** are siblings of parent node **B**
  - **K, L**, and **M** are siblings of parent node **I.**

– **Descendents:** The **descendents** of a node consist of its children, and their children, and so on. All nodes in a tree are descendents of the root node (except the root node itself).

– **Ancestors:** The **ancestors** of a node consist of the parent node and the parent of that node also. The root node is the ancestor of all nodes and the root does not have any ancestor.

– **Order (Degree): Order (degree) of a node** is given by the number of its children. If a node has two children, its degree is two.
In the above tree.
  - degree of node A is 3.
  - degree of node B is 2.
  - degree of node C is 1.

Similarly, **order (degree) of a tree** is given by the maximum of the degrees of its nodes. In other words, if the degree of a tree is n, its nodes can have, at most, n children. In the above tree the node A has degree 3 which is the maximum degree. **So the degree of the above tree is 3**.

– **Edge:** The link between any two nodes is called an **edge**. It is directed from the parent towards its child.

– **Path:** There is a **single, unique path** from the root to any node. For e.g. the path from node A to J is A, B, F, and J.

– **Height:** A **height of a node** is equal to the maximum path length **from that node to a leaf node**. A leaf node has a height of 0. The **height of a tree** is equal to the height of the **root**. For e.g. the height of node B from leaf node J is 2.

- **Depth:** A **depth of a node** is equal to the maximum path length from root to that node. A root has a depth of 0. **The depth of a tree** is equal to the depth of the deepest node in the tree. It, of course, is equal the tree height. For e.g. the depth of node J is 3.

- **Forest:** it is a set of disjoint tress. In a given tree, if you remove its node then it becomes a forest.

- **Level:** The **level of a node** is 0, if it is root. Otherwise, it is one more than its parent. For e.g.
  - Node **C** is at level 1
  - Node **K** is at level 3

Binary Tree

A binary tree is a finite set of data items which is either empty or consists of a single item called the root and the two disjoint binary trees called the **left subtree** and **right subtree.** In a binary tree the maximum degree of any node is **at most two.** That means, there may be a zero degree node (usually an empty tree) or one degree node or two degree node.

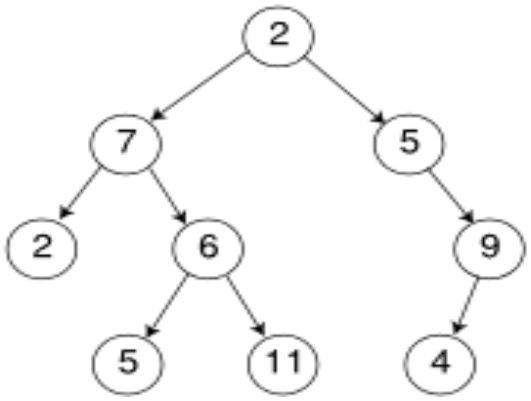


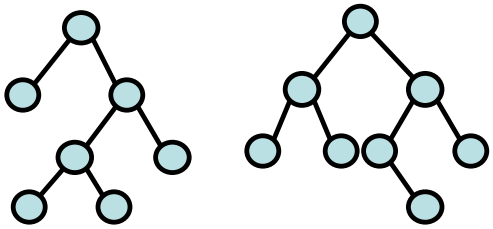
Fig: A simple binary tree, in which every node has at most two children.

Some properties of a binary tree

- Number of external nodes ≤ number of internal nodes + 1
- Number of nodes at level i $\leq 2^i$
- Number of external nodes $\leq 2^{height}$ (Here, height means height of the tree)
- Number of nodes in a tree $\leq 2^{height+1}-1$ (Here, height means height of the tree)
- height $\geq \log_2$(number of external nodes)
- height $\geq \log_2$ (number of nodes) – 1

Types of binary trees
There are two types of binary trees.

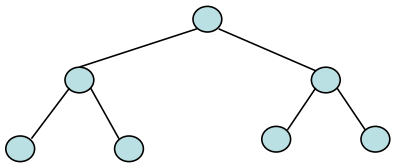
A **Strictly binary tree** or **Full binary tree**, or **Proper binary tree**, is a tree in which every node has zero or two children. i.e. Every internal node has nonempty left and right subtrees. A strictly binary tree with n leaves contains 2n – 1 nodes.



Here the first tree is a Strictly binary tree, while the second one is not.

A **Complete binary tree** or a **Perfect binary tree** is a strictly binary tree in which all leaves are at the same depth. It has following properties.

– Every internal node has 2 children
– Height of n will have $2^{n+1} - 1$ nodes
– Height of n will have 2^n leaves
– No of nodes in level l have 2^l nodes
– No of external nodes = No of internal nodes + 1



Tree traversals

Visiting the nodes in certain order is called traversing. The reason for visiting the nodes is

to process the contents of the node according to our requirement.

In a linear structure like an array or a linked list, there are two ways of traversal
–from front to end and from end to front
These traversals can be achieved, simply, by using loops.

Tree, being a non-linear data structure, there are three different ways for traversal.
Preorder, Inorder and Postorder traversals.

The general rules for these traversals are as follows:

PreOrder traversal (also called depth-first traversal)

In this traversal, the root node is visited prior to the left or right node in a subtree. The rule is

1. Visit the root node (**N**).
2. then, visit the nodes in the left subtree in preorder(**L**).
3. finally, visit the nodes in the right subtree in preorder(**R**).

InOrder traversal

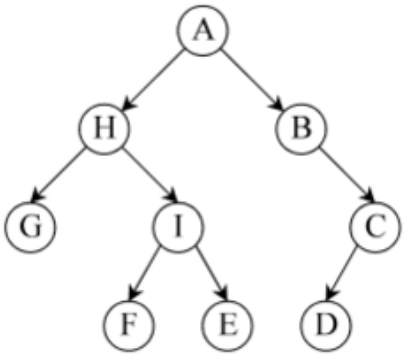
In this traversal, the root node is visited in between the left and the right node in a subtree. The rule is

1. Visit the nodes in the left subtree in inorder (**L**).
2. Visit the root node (**N**).
3. Finally, visit the nodes in the right subtree in inorder(**R**).

PostOrder traversal (also called breadth first traversal)

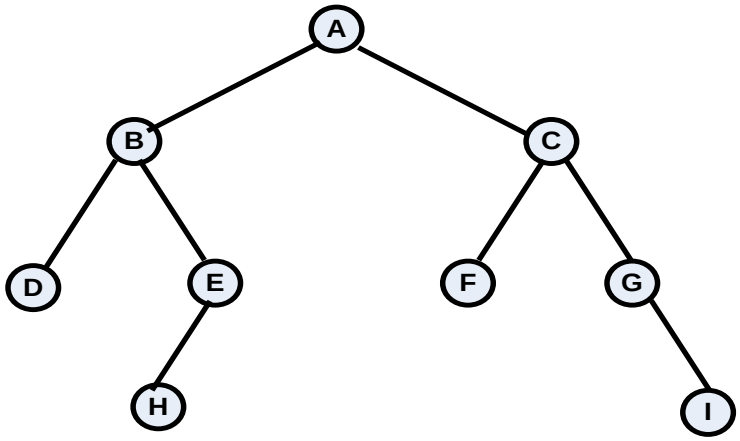
In this traversal, the root node is visited after the left and the right node in a subtree. The rule is

1.visit the nodes in the left subtree in postorder(**L**)
2.then, visit the nodes in the right subtree in postorder(**R**).
3.Finally, visit the root node (**N**).



- Preorder traversal : A, H, G, I, F, E, B, C, D
- Postorder traversal : G, F, E, I, H, D, C, B, A
- In-order traversal : G, H, F, I, E, A, B, D, C

More examples of tree traversal



The order of **Preorder traversal** is: *A B D E H C F G I*

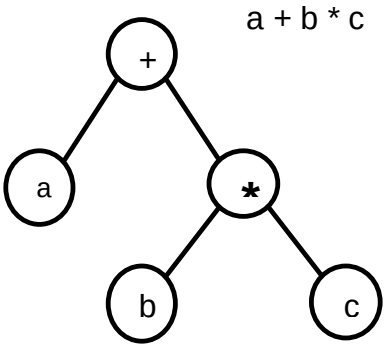
The order of **Inorder traversal** is: *D B H E A F C G I*
The order of **Postorder traversal** is : *D H E B F I G C A*

Expression Tree

An expression tree is used to represent arithmetic and logical expressions. It is built up from the simple operands and operators of an expression by placing the simple operands as the leaves of a binary tree, and the operators as the interior nodes.
For each binary operator, the left subtree contains all the simple operands and operators in the left operand of the given operator and the right subtree contains everything in the right operand.
For a unary operator, one subtree will be empty.

Expression tree traversals

When an expression tree is traversed in inorder, we get an infix expression. When it is traversed in preorder, we get the equivalent prefix expression. Similarly, when it is traversed in postorder, we get the equivalent postfix expression.

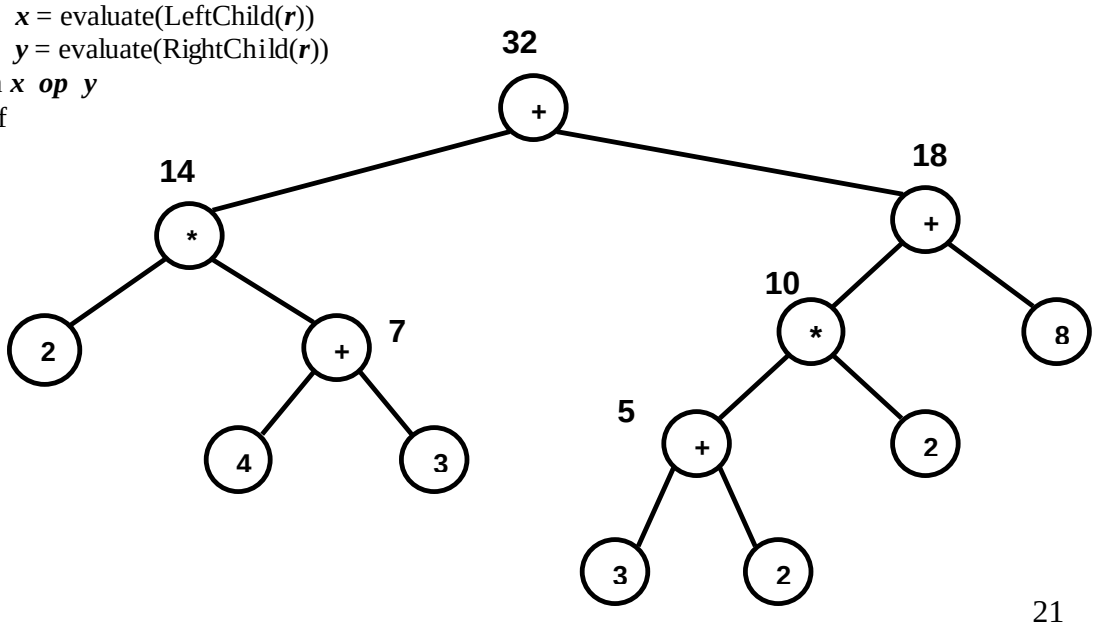


Evaluation of Expression Tree

Evaluation of an expression tree is done by applying the operator at the root to the values obtained by recursively evaluating left and right subtrees

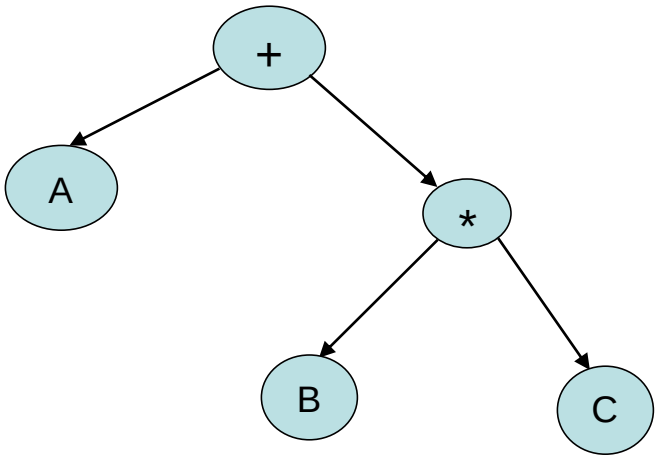
An Algorithm

evaluate (***r***)
– If ***r*** is a leaf
return the value stored in ***r***
– Else
Let op be the operator stored in ***r***
r=*leftchild(**r**)*
x = evaluate(LeftChild(***r***))
y = evaluate(RightChild(***r***))
return ***x op y***
– End If



Example of an expression tree from a given expression

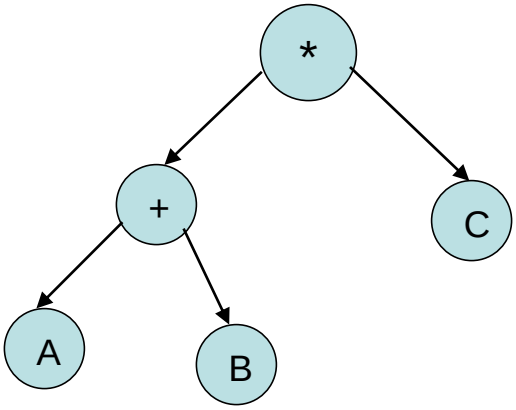
1. A + B * C



As, already said, when an expression tree is traversed in inorder, we get an infix expression. When it is traversed in preorder, we get the equivalent prefix expression. Similarly, when it is traversed in postorder, we get the equivalent postfix expression. So, now let us check:

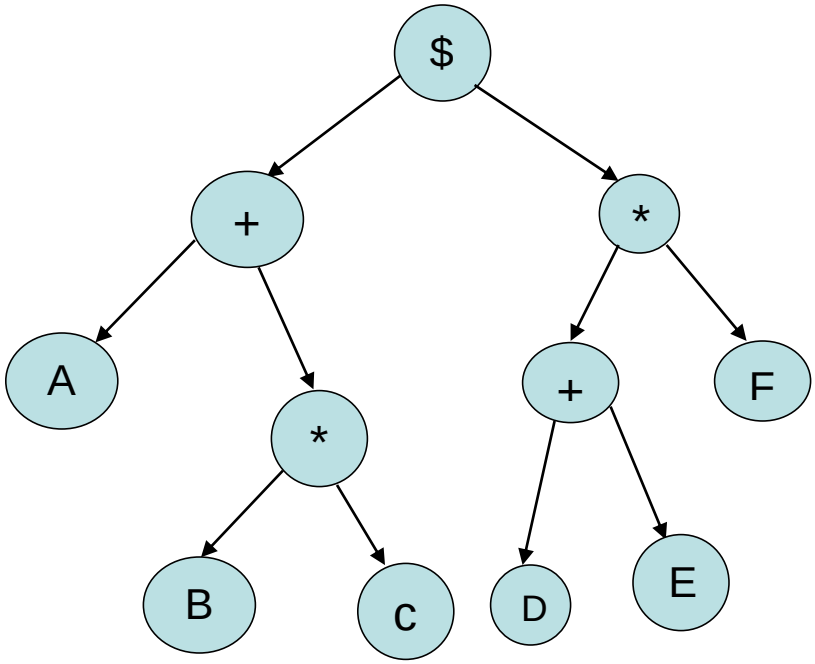
The preorder traversal is: + A * B C (equivalent to prefix expression)
The postorder traversal is: A B C * + (equivalent to postfix expression)

2. (A + B) * C



The preorder traversal is: * + A B C (equivalent to prefix expression)
The postorder traversal is: A B + C * (equivalent to postfix expression)

3. (A + B * C) $ ((D + E) * F)



The preorder traversal is: $+A*BC*+DEF (equivalent to prefix expression**)**
The postorder traversal is: ABC*+DE+F*$ (equivalent to postfix expression**)**
Algorithm for construction of an expression tree out of a postfix expression

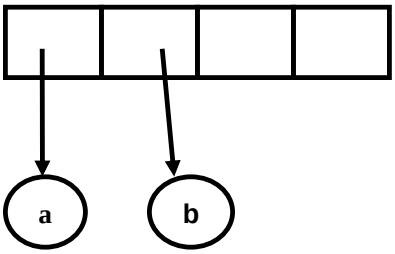
Given a postfix expression, following is the algorithm to build an expression tree. The inorder and preorder traversals of this tree will give infix and prefix expressions respectively.

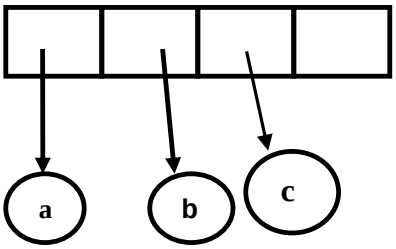
1. Make an empty stack.

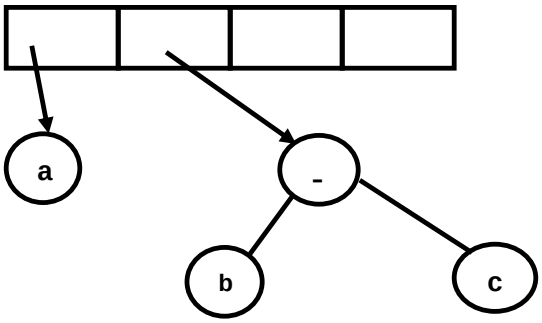
2. For each character from left to right
- if the character is an operand.
    make a single node tree for this operand, push its address into the stack.
  elseif it is an operator, say op
    make a single node tree for this operator
    pop the stack, make it right child of op
    pop the stack, make it left child op
    push the tree (whose root is op) into the stack
- endif

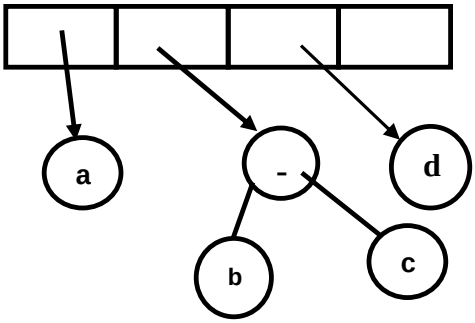
3. Pop the stack to get the required expression tree

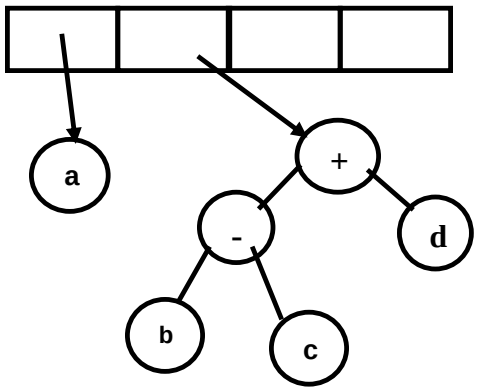
Let the postfix expression be: **abc-d+***

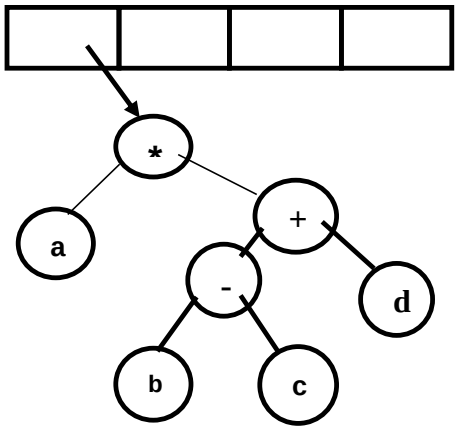

a
b


a
b
c


a
-
b
c


a
-
d
b
c


a
+
-
d
b
c


*
a
+
-
d
b
c

Representation of Binary Tree

A binary tree can be represented in two different ways.

- Using linked list
- Using array

Linked representation and implementation

A binary tree is a collection of nodes. Each node in a binary tree has three fields, the data part, and two pointers that point to the root node of left and right subtree. For a leaf the left and right pointers will be null. Also for an empty subtree, the corresponding pointers will be null. An external pointer is used to point to the root node of the tree.

```
struct bnode
{
        int info;                   //for storing data. Currently it is of type int.
        struct bnode *leftchild;    //pointer to the root of the left subtree
        struct bnode *rightchild;   //pointer to the root of the right subtree
};
```

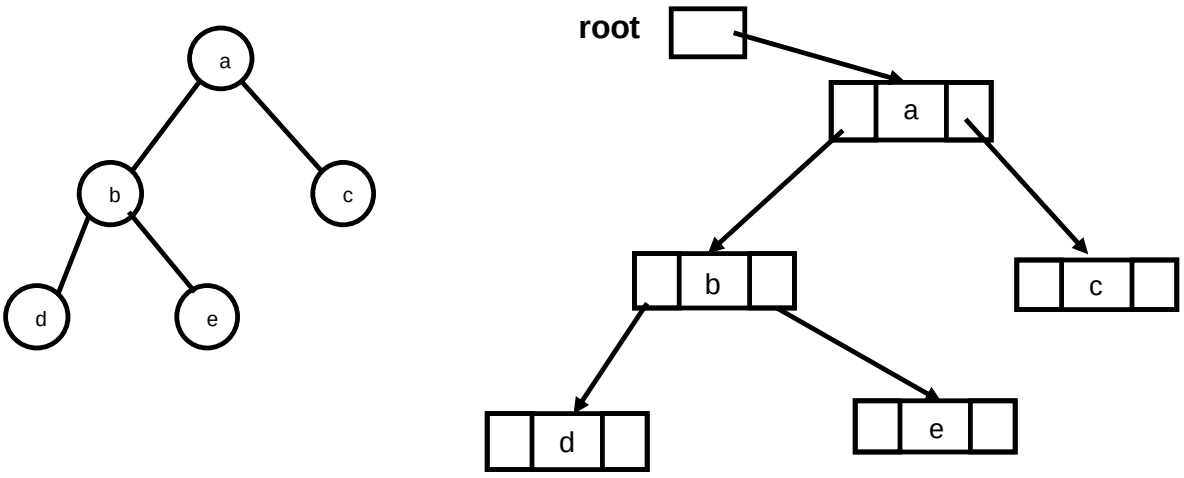


Fig: A binary tree and its linked list representation

Array Representation of Binary Tree

A Binary Tree can be represented by means of a linear array having index ‘k’, 0< k<= n, where n is the number of nodes.

The root node is stored at the index 0.
Parent(k) : The Parent of the node at index k is given by floor((k-1)/2) if k is not equal to 0

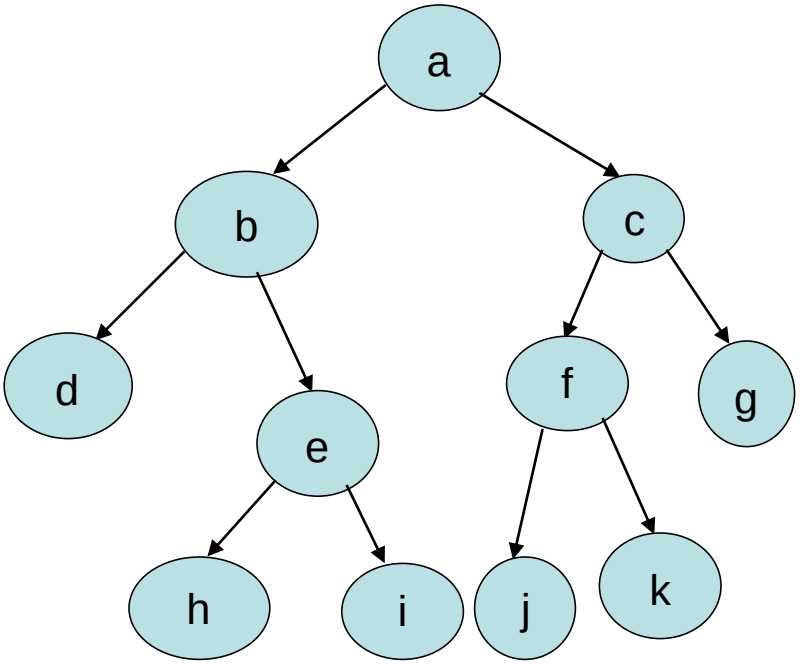
LeftChild(k) : Left child of the node at k is given by 2k+1.

RightChild(k) : Right child of the node at k is given by 2k+2.

Siblings(k) : If the left child at index k is given, then its right sibling is at k+1. Similarly, if right child at index k is given, its left child is at k-1.

Some Binary Trees along with their sequential representation are as follows:

0	**a**
1	**b**
2	**c**
3	**d**
4	**e**
5	**f**
6	**g**
7	
8	
9	**h**
10	**i**
11	**j**
12	**k**
13	
14	



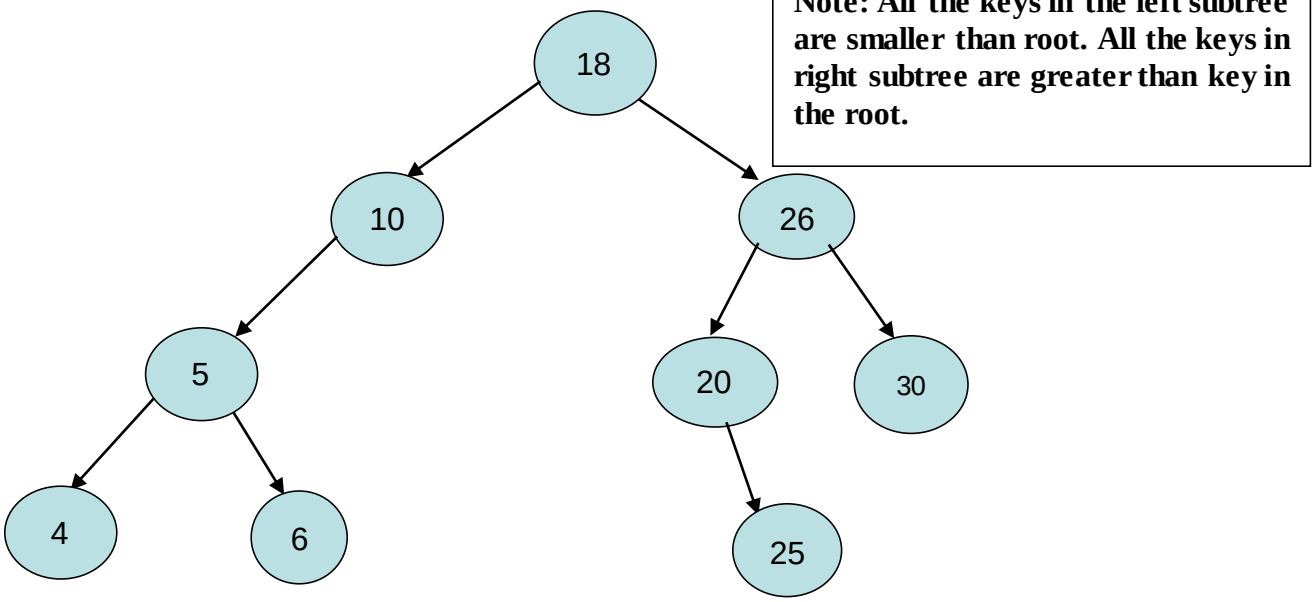
A sequential representation consumes more space for representation binary trees. But for representing complete binary tree, it proves to be efficient as no space is wasted. But, the amount of spaces has to be determined at first.

Binary Search Tree (BST)

A Binary Search Tree is a binary tree which is either empty, or in which every node contains a key and satisfies the following criteria:

- All keys (if any) in the left –subtree is smaller than the key in the root.
- All keys (if any) in the right subtree is larger than the key in the root.
- The left and right subtrees again BST.

E.g. 18 10 26 5 20 30 4 6 25



Note: All the keys in the left subtree are smaller than root. All the keys in right subtree are greater than key in the root.

- With the above definition, we can also conclude that, no two nodes in a BST have same value.
- The inorder traversal of a BST yields the keys in a sorted order (ascending).
- If the tree is reasonably full, searching for a key takes **$O(\log_2(n))$** time, where n is the number of nodes.

Operations on a Binary Search Tree

Search(x, T): Search for key x in the BST T.if x is found in some node n of T return a pointer to node n or NULL otherwise.

Insert (x,T): Insert a new node with x in the info part in the tree T such that the property of BST is maintained.

Delete(x,T): Delete a node x in the info part from the tree T such that the property of BST is maintained.

FindMin(T): Returns pointer to a node having minimum key in tree T,NULL if T is empty.

FindMax(T): Returns pointer to a node with maximum key in T,NULL if T is empty.

Searching

In BST, searching of the desired data item can be performed by branching into left or right subtree until the desired data item is found. We proceed from the root node.

- If the tree is empty, then we failed to find x, so return NULL.
- Else, compare x with the key stored in the root of T.
  - ✓ If x is same as the key in root, return address of the root.
  - ✓ If x is smaller than the key in root, search for x in the left subtree of T recursively.
  - ✓ If x is greater than the key in root, search for x in right subtree recursively.

Insert

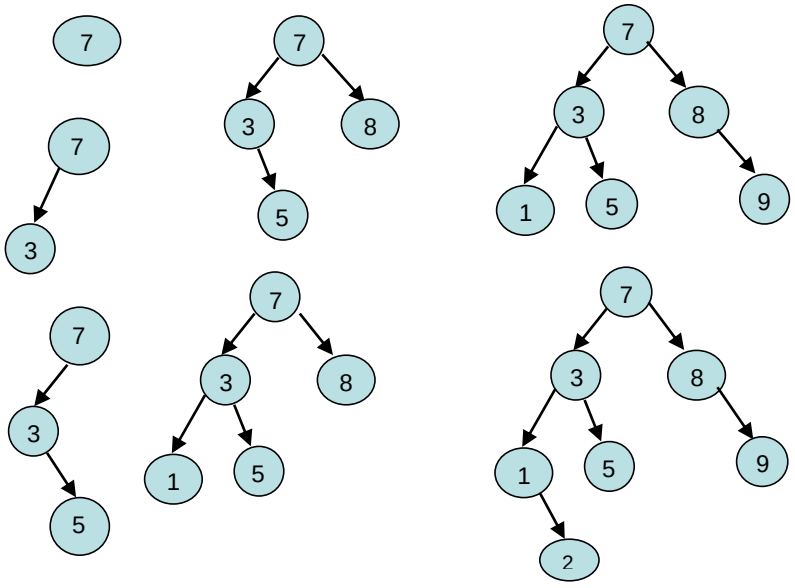
In BST, we don't allow duplicate data items. So, to insert a data item having key 'K' in a binary search tree, we must check that its key is different from those of existing data items by performing a search for data item with the same key K. If the search is unsuccessful, the data item is inserted into BST at the point the search is terminated.

Algorithm

Given a new item with key x, then following algorithm will insert it in BST T.

- If the tree is empty, create a new node n storing the key x and make this new node n the root of the tree. Otherwise, compare x to the key stored at root R.
- If x is identical to key in R, don't change the tree.
- If x is smaller, insert new item into left subtree of the root recursively.
- If x is larger, insert new item into right subtree of the root recursively.

E.g. Trace 7,3,5,8,1,9,2



Now trace 15,10,20,17,25,5,13,33,9,21

Deletion

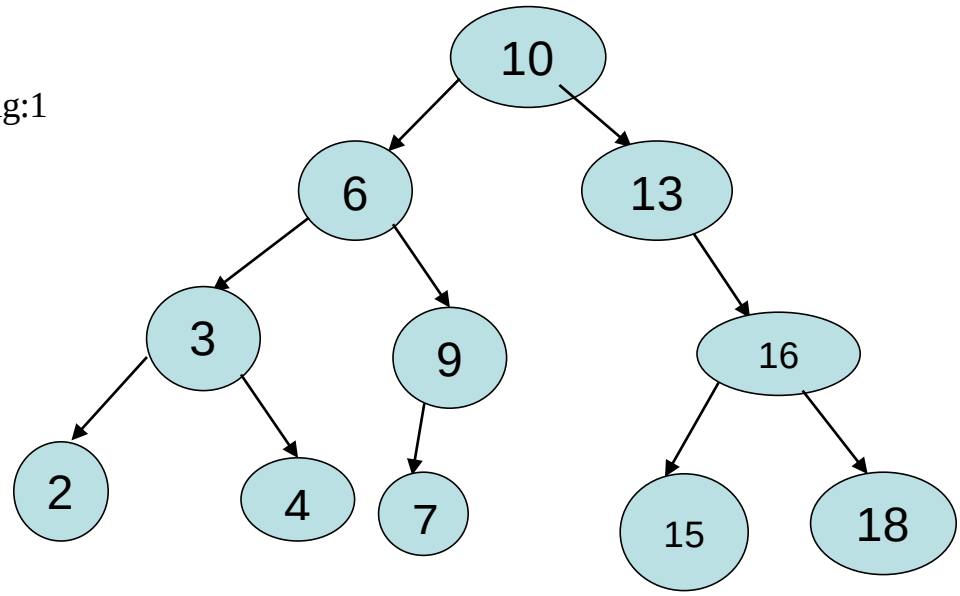
To delete a given node (key), it is searched first. If search is unsuccessful, the algorithm terminates. Otherwise, the following possibilities arise:

- If node is a leaf, it can be deleted immediately.
- If the node has one subtree, the node can be deleted by replacing it with the root of the non-empty subtree.

If the node has two subtrees, we replace the node with the rightmost node of the left subtree T_r or the leftmost node of the right subtree T_r

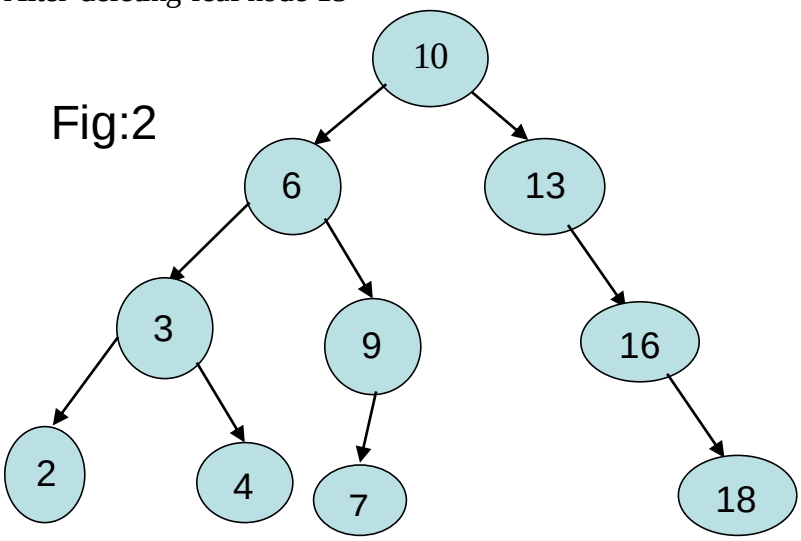
E.g.

Fig:1



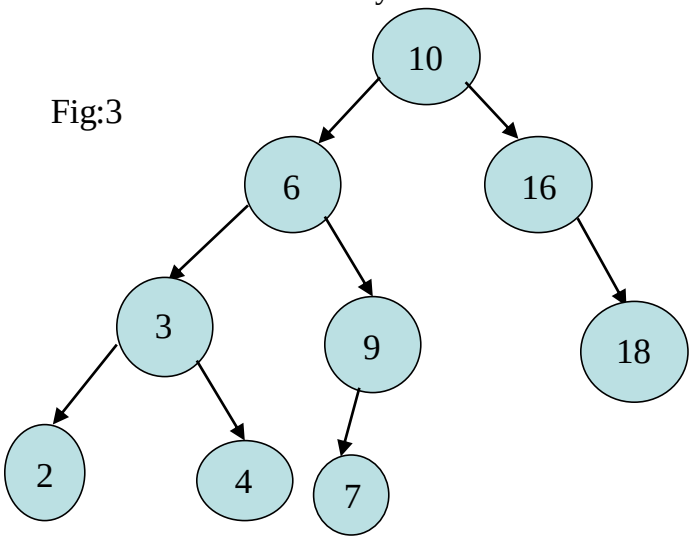
After deleting leaf node 15

Fig:2

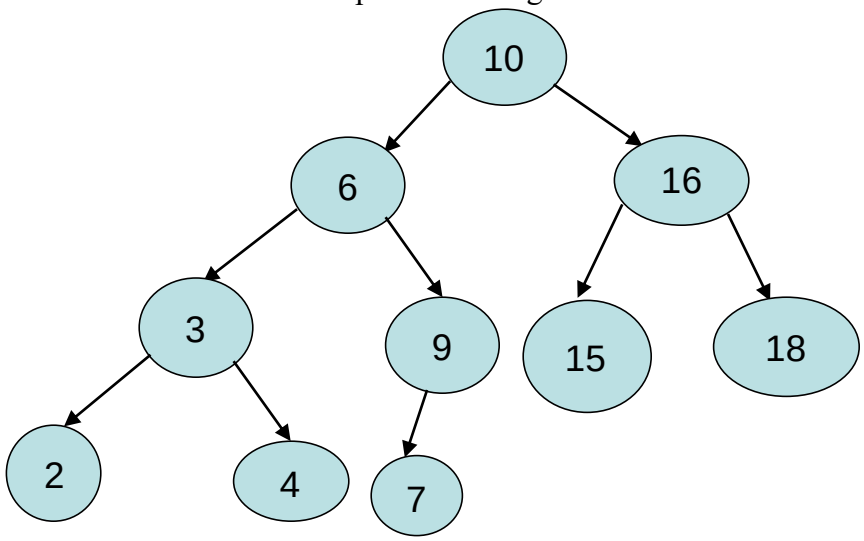


Now lets delete node with key 13 from the above tree.

Fig:3

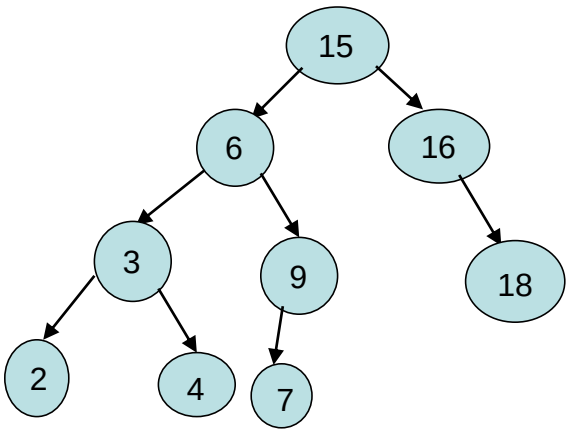


Now let's see another example of deleting root node.

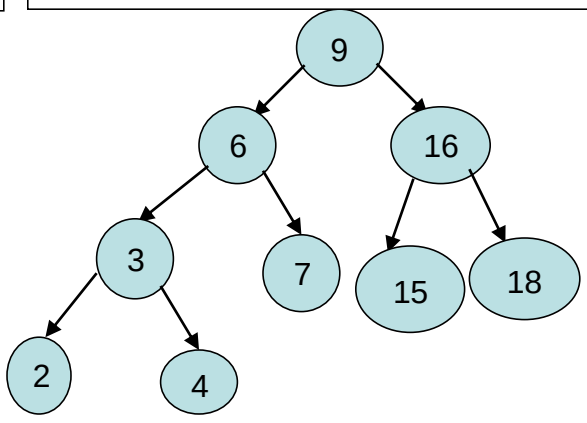


Deleting root node with key 10

Leftmost node of right subtree takes its place



Rightmost node of left subtree takes its place

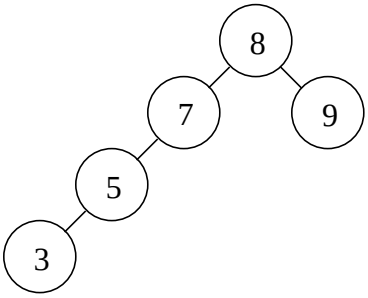


Heap

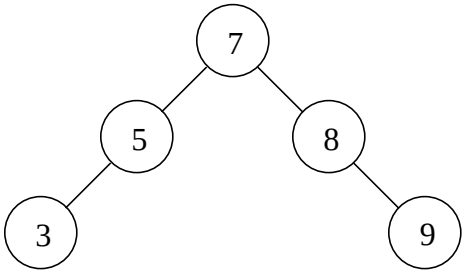
A **binary heap** or simply a heap is an **almost complete binary tree** with the key in each node, such that

- All the leaves of the tree are on two adjacent levels.
- All the leaves on the lowest level occur to the left and all levels except possibly the lowest are filled.
- The key in the root is at least as large as the keys in its children (if any), and the left and rights subtrees are again heaps.

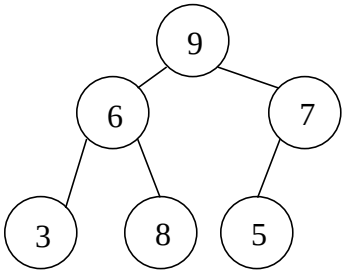
Above definition is of max heap (descending partially ordered tree). For min heap the root has the smallest key.



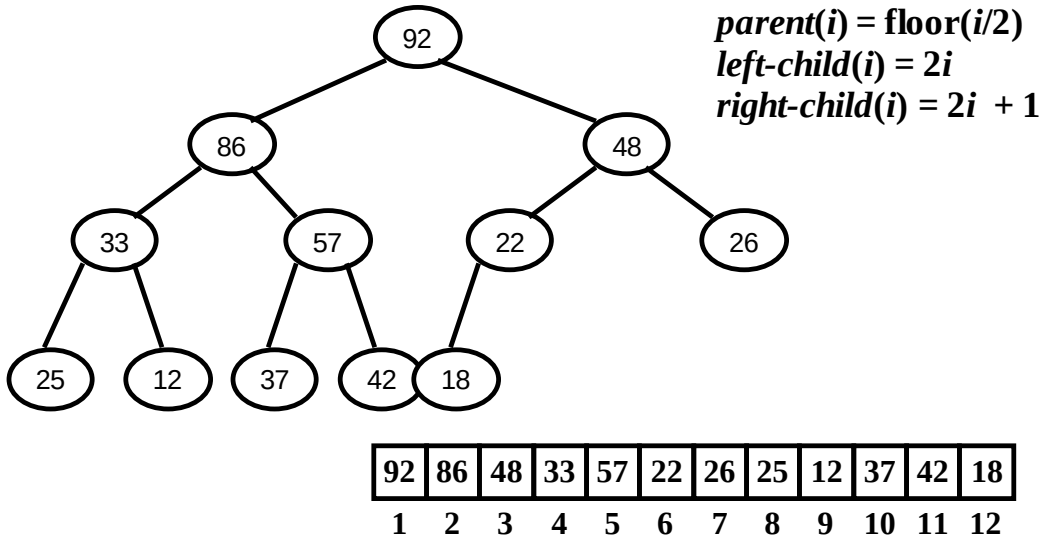
This is not a Heap since it violates rule 1



This is not a Heap since it violates rule 2



This is not a Heap since it violates rule 3



Note that a heap is definitely not a search tree

Types of Heap:

Max Heap : For max heap, the root has the largest key. The key at the parent node is larger than the keys at their children and descendents.

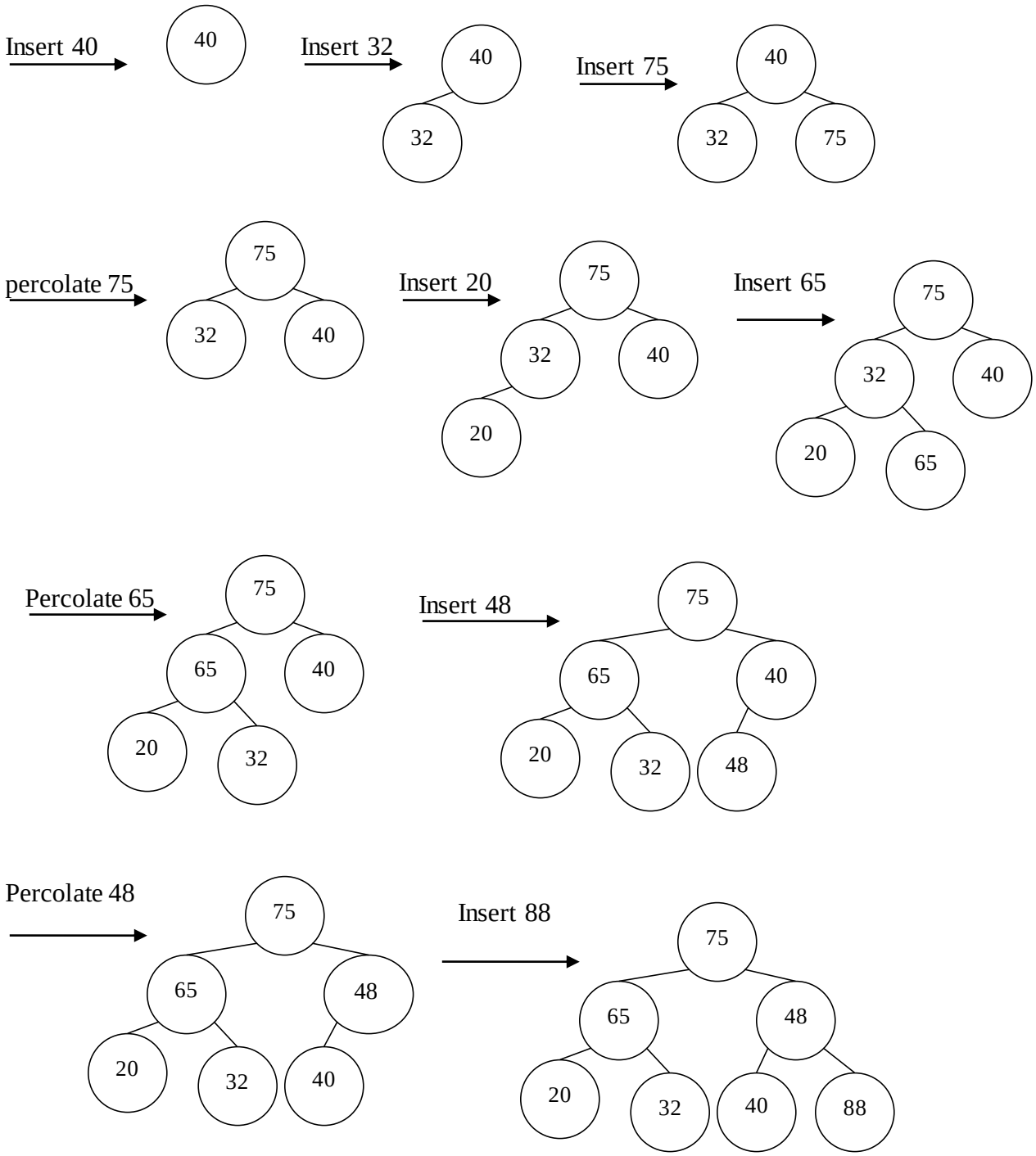
Min Heap : The root has the smallest key. The key at the parent node is smaller than the keys at their children and descendents.

Heaps are represented in array rather than linked structures since it is a special kind of binary tree that have no gaps in an array implementation.

Creating a Heap and Insertion:

A Binary tree with only one node satisfies the properties of a heap.
To insert a new element in to the heap, we create a hole in the next available location, since otherwise the tree will not be complete tree.
If new item can be placed in the hole without violating heap order, then we do insert there. Otherwise, we slide the element that is in the hole's parent node into the hole, thus bubbling the hole up towards the root. The process is continued until the new item can be placed in the hole. This strategy is known as a percolate up. The new element is percolated up the heap until the correct location is formed.

Eg: Inserting following numbers in the max Heap: 40, 32, 75, 20, 65, 48, 88. 7



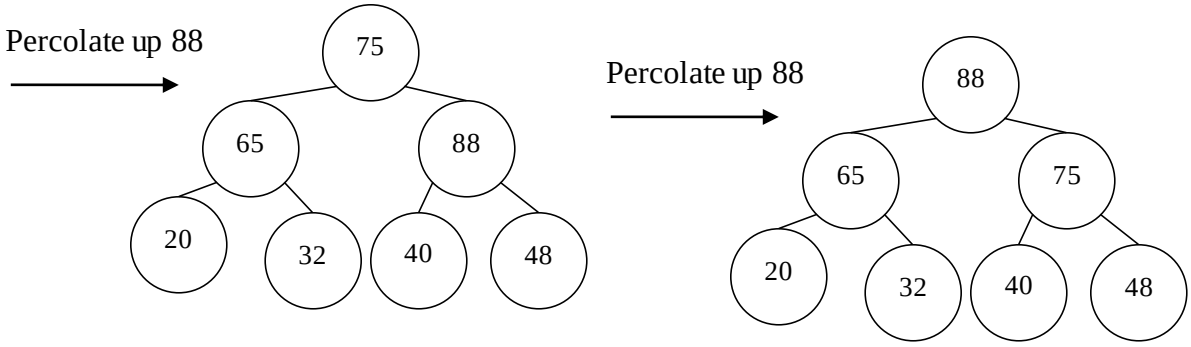


Fig: Creating Max Heap

B-Tree

In binary search tree, each node contains a single key and points to two sub-trees. We may extend this concept to a general search tree in which each node contains one or more keys.

A multi-way search tree of order n is a general tree in which each node has n or fewer sub-trees and contains one fewer key than it has sub-trees.

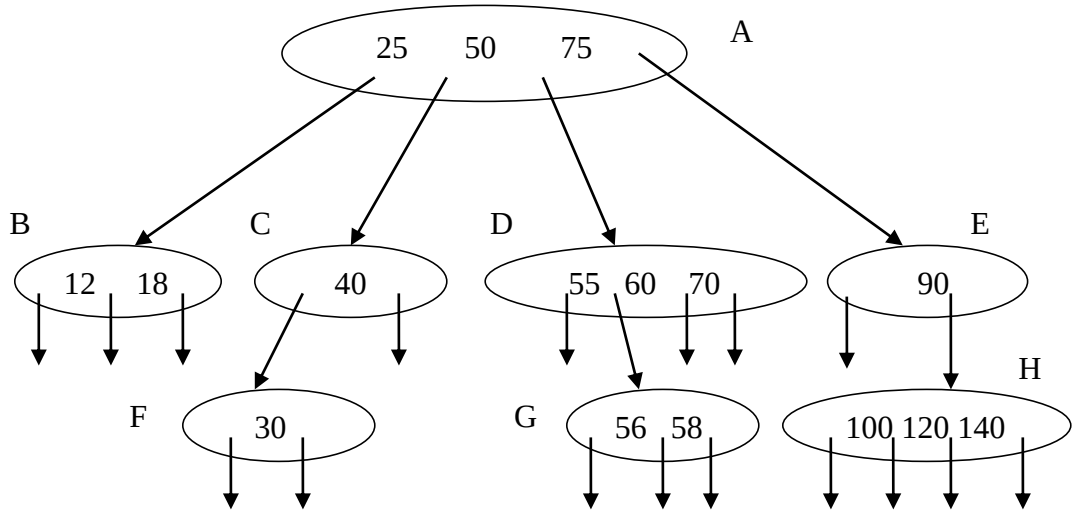


Fig: A multiway search tree of order 4

Each node can have at most 3 keys and 4 children, since the order is 4. The nodes A, D and H are called full nodes since it has the maximum no. of keys.

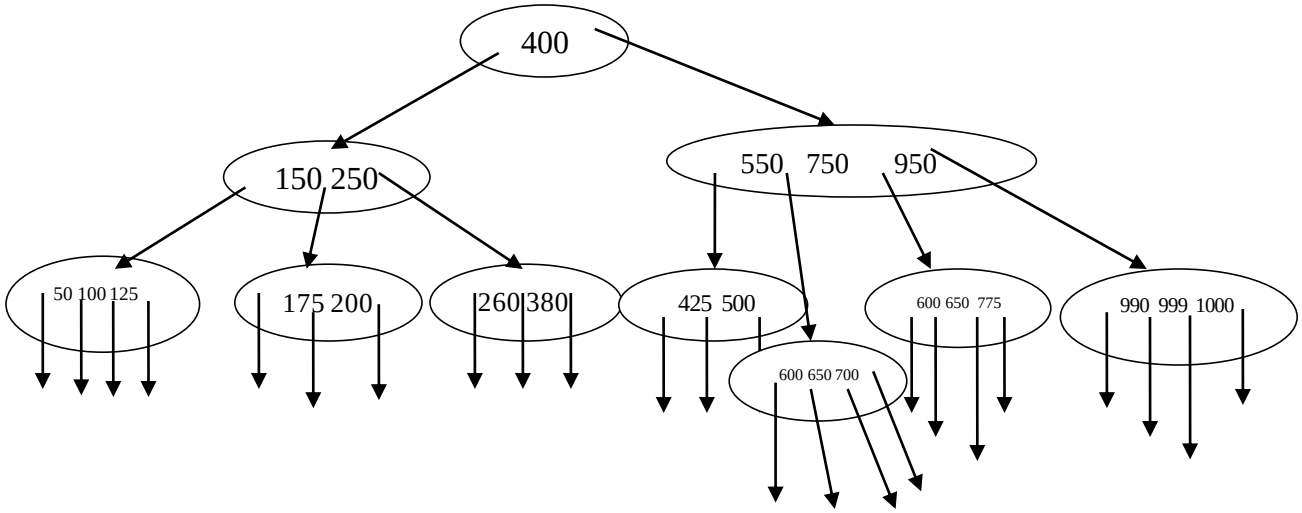
Balanced – multiway Tree

A multiway search tree can be improved to minimize accesses by accomplishing the followings:

1. make the height of the tree as small as possible
2. no empty subtrees appear above the leaves (so that the division of keys into subsets is as efficient as possible).
3. all the leaves be on the same level (so that searches will be guaranteed to terminate with about the same number of accesses).
4. every node (except the leaves) have at least some minimal number of children. These conditions leads us to the definition of B-Tree.

Definition of B-Tree

A B-tree of order m is an m-way search tree in which
1. All leaves are on the same level
2. All internal nodes except the root have at most m nonempty children, and at least (m/2) nonempty children
3. The number of keys in each internal node is one less than the number of its children, and these keys partition the keys in the children in the fashion of a search tree.
4. The root has at most m children, or as few as 2 if it is not a leaf, or none if the tree consists of the root alone



Insertion into a B-Tree

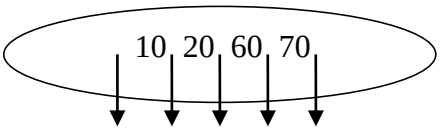
The condition that all the leaves be on the same level forces a characteristic behavior of B-Tree. In contrast to binary search tree, B-Tree are not allowed to grow at their leaves, instead they are forced to grow at the root. The general method of insertion is as follows.

First a search is made to see if the new key is in the tree. This search (if the key is new) will terminate in failure at a leaf. The new key is then added to the leaf node. If the node was not previously full, then the insertion is finished.

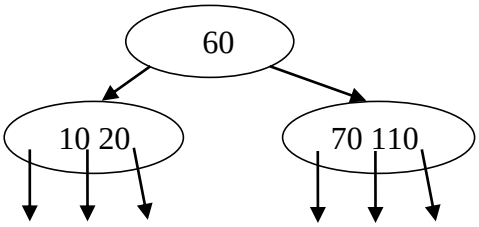
When a key is added to a full node, then the node splits into two nodes on the same level, except that the median key is not put into either of the two new nodes but is instead sent up the tree to be inserted into the parent node.

Illustration: B-Tree of order 5 is as follows:
10, 70, 60, 20, 110, 40, 80, 130, 100, 50, 190, 90, 180, 240

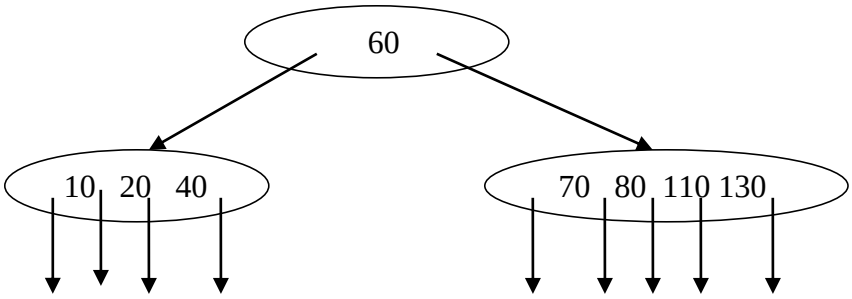
Inserting 10, 70, 60, 20



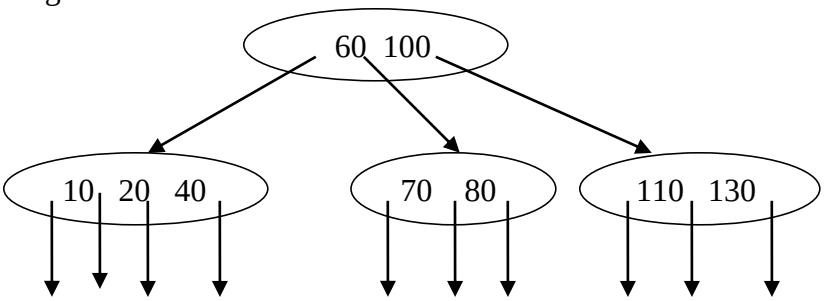
Inserting 110



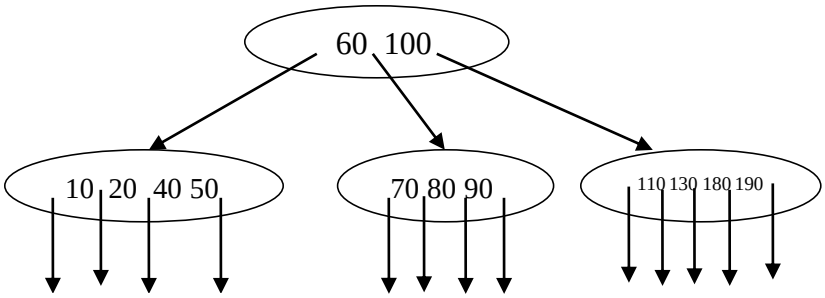
Inserting 40, 80, 130



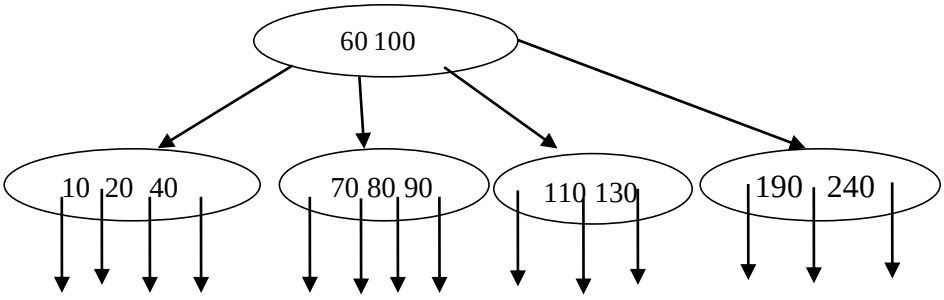
Inserting 100



Inserting 50, 190, 90, 180



Inserting 240



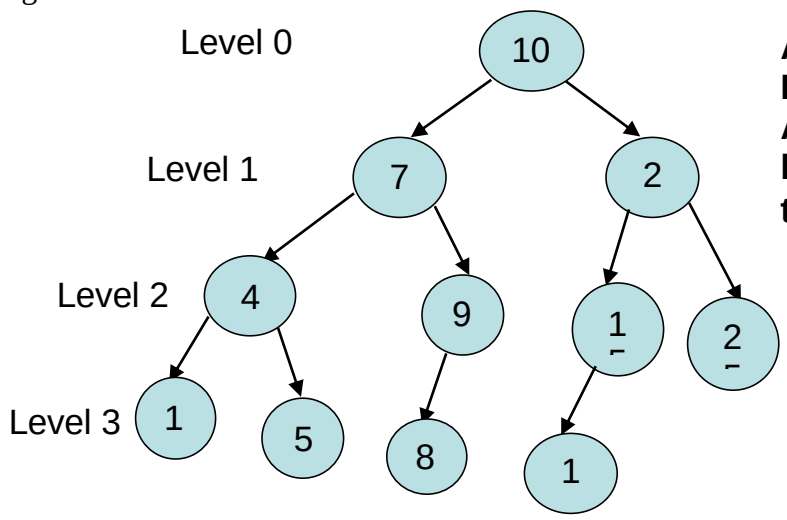
The first four keys will be inserted into one node as shown in the diagram. They are sorted into proper order as they are inserted. There is no room for fifth key 110, so its insertion causes the node to split into two, and the medial key, 60 moves up to enter a new node, which is a new root. Since the split nodes are now only half full, the next three keys can be inserted without any difficulty. Note these simple insertions can require rearrangement of the keys within a node. The next insertion 100, again splits a node and this time it is 100 itself that is the median key, so moves up to join 60.

Next several insertions are similar. The final insertion of 240 splits the node containing 110,130, 180 and190 which is again full. Hence, this node in turn splits and 180 is promoted into the root.

Try inserting 30, 120, 140, 200, 210, 160 respectively into the above B-Tree.

Height Balanced Trees or AVL Trees

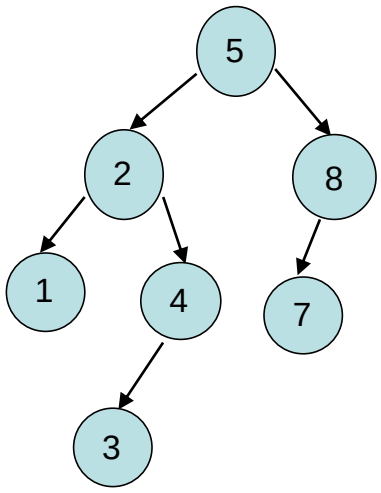
A binary tree of height h is completely balanced if all the leaves occur at level h or h-l, and all nodes at levels lower than h-l have two children.
E.g. of balance tree



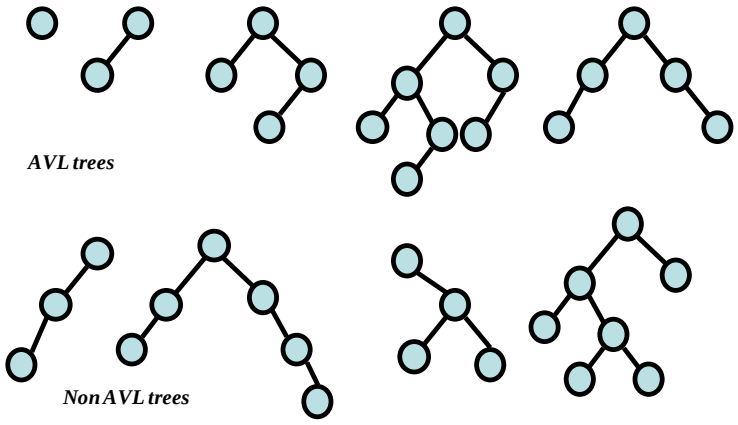
Definition: An AVL tree is a binary search tree in which the heights of the left and right subtree of the root differ by, at most, 1 and in which the left and right subtree are again AVL trees. It is named after their inventors' G.M. Adelson, Velskii and E.M. Landis.

Insertion in AVL Tree
A new node is inserted in an AVL tree by using usual binary search tree algorithm. It, often, is inserted without changing the height of the subtree. Even when the height of the subtree increases, it may be shorter subtree that has grown.
But there may be cases where, a node is added to a subtree that is taller than other subtree. Then, the height difference would be 2.

If this is the case, then the property has to be restored before the insertion step is considered over.
We use 'ROTATIONS' to maintain the AVL Tree properties.
E.g.



AVL tree property will be violated at node 8 if we insert 6.



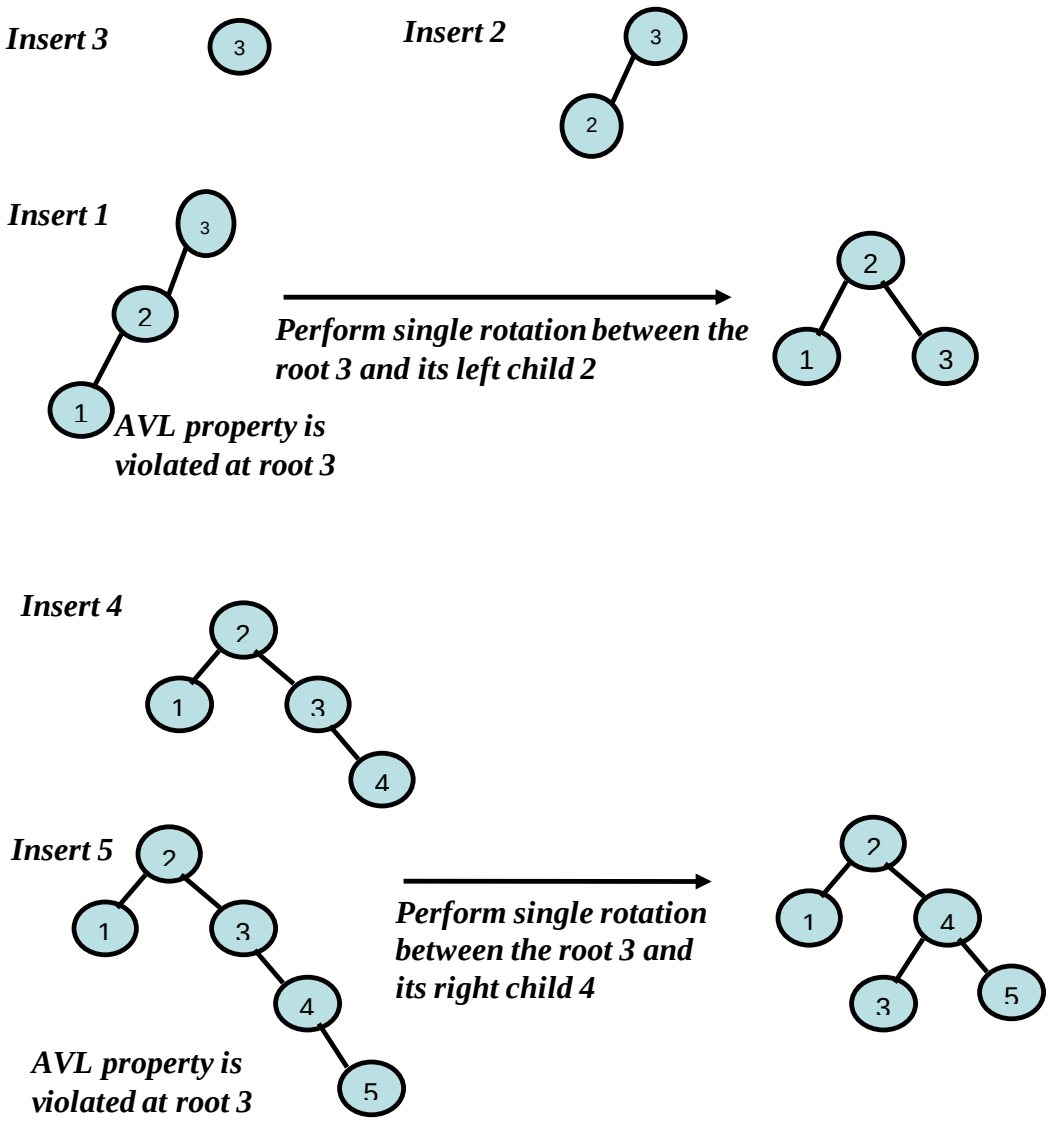
Cases of violation
After insertion, violation might occur in four ways.

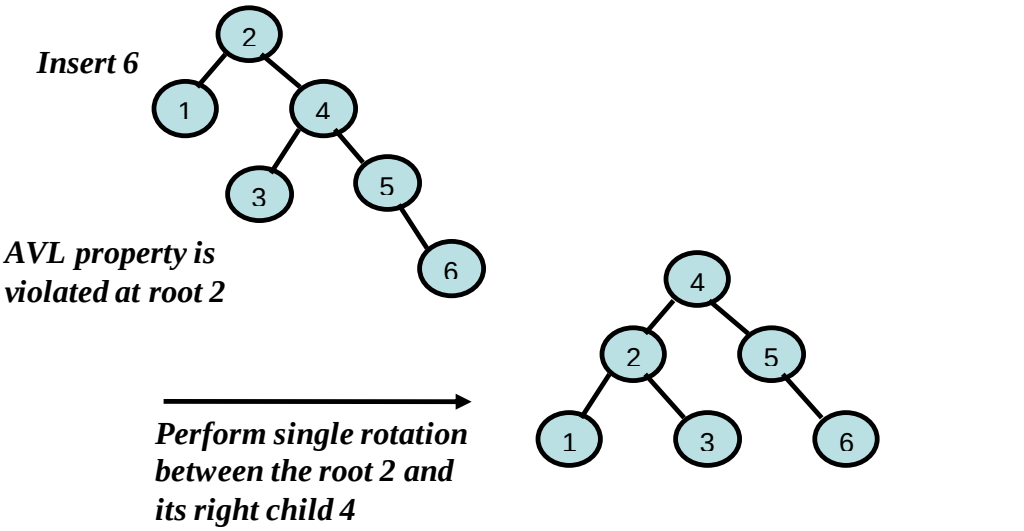
1) Insertion at left subtree of the left child of node n.
2) Insertion at right subtree of the left child of node n.
3) Insertion at left subtree of the right child of node n.
4) Insertion at right subtree of the right child of node n.

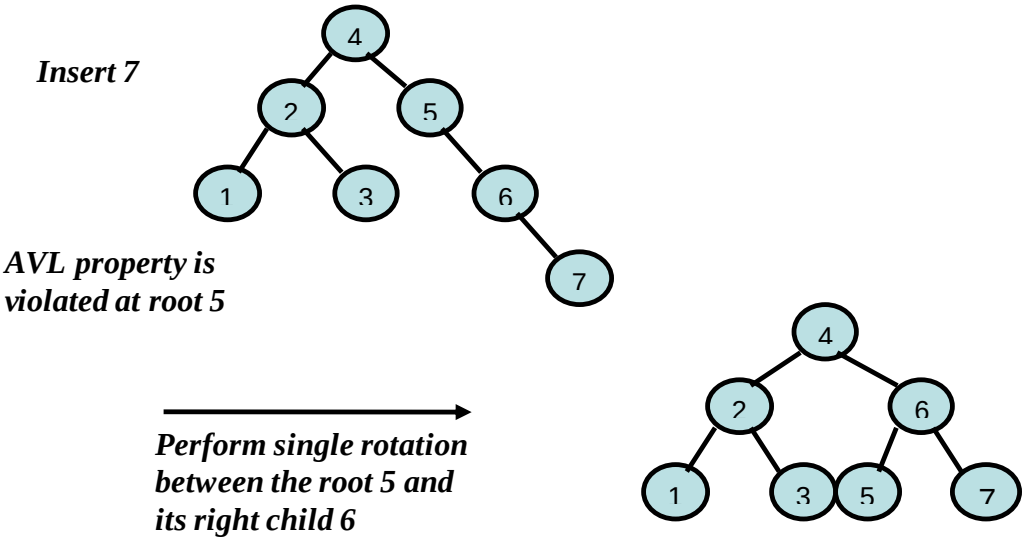
The first and forth cases (left-left and right-right) is rebalanced using Single Rotation.
The second and third cases (right-left and left-right) are fixed using Double Rotation.
This is also known as dogleg pattern.

Tracing of Single Rotation:

Insert 3, 2, 1, 4, 5, 6, 7 in an empty AVL tree

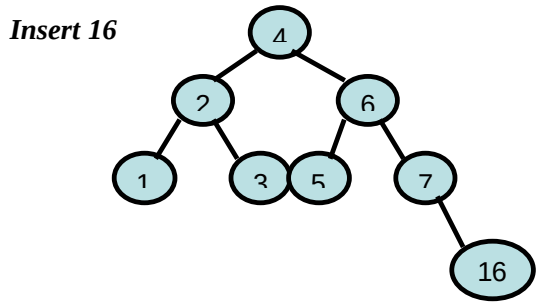


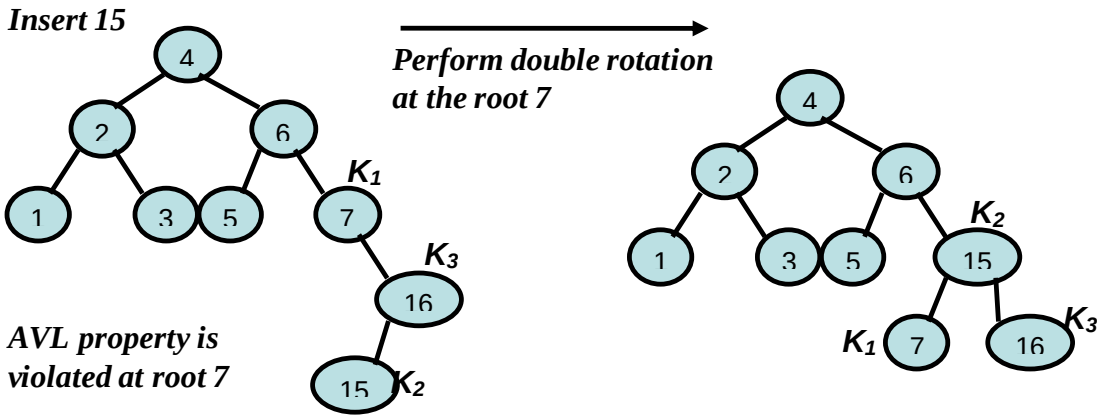


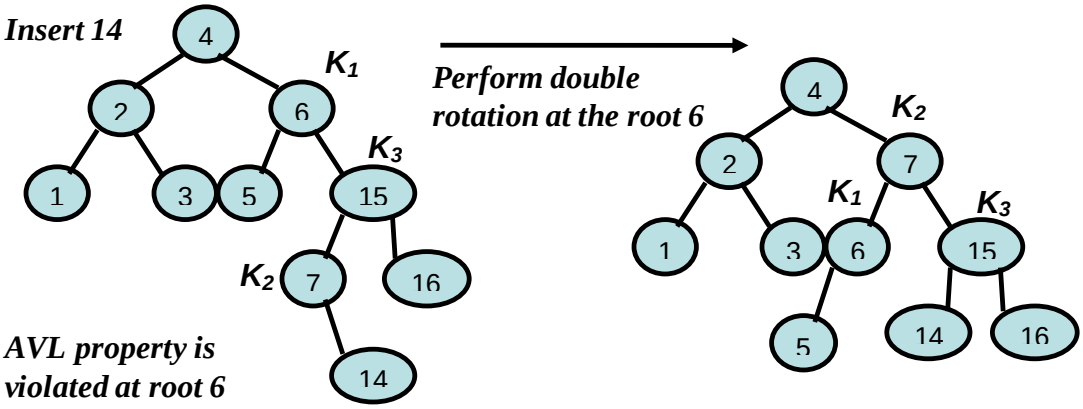


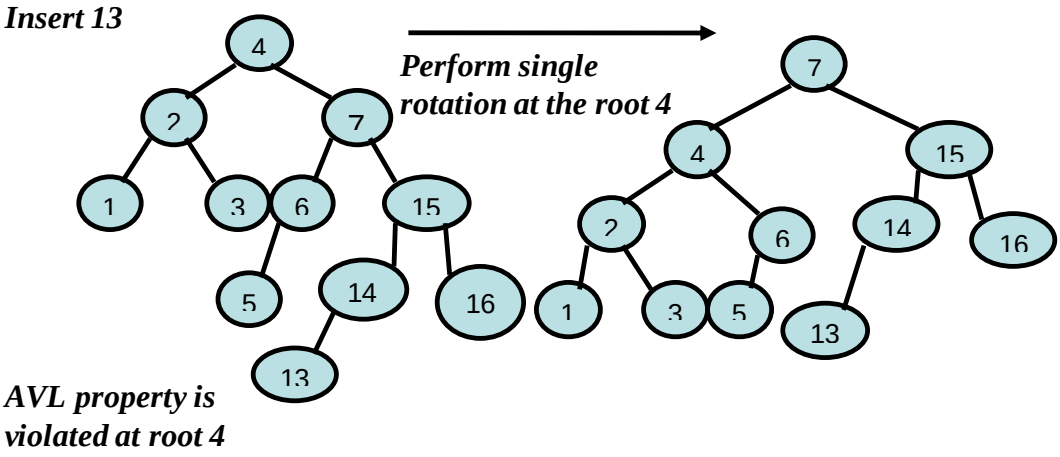
Tracing of Double Rotation:

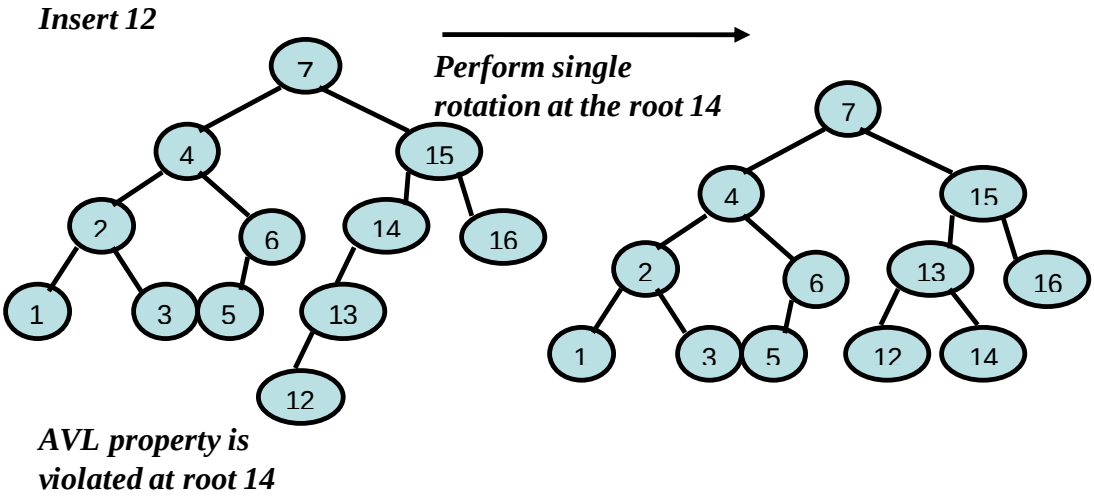
Continue inserting 16,15,14,13,12 in the previous tree






Insert 14
4
2
6
1
3
5
15
7
16
14
K1
K3
K2
AVL property is violated at root 6
Perform double rotation at the root 6
4
2
7
1
3
6
15
5
14
16
K2
K1
K3


Insert 13
4
2
7
1
3
6
15
5
14
16
13
AVL property is violated at root 4
Perform single rotation at the root 4
7
4
15
2
6
14
16
1
3
5
13



Deletion in AVL Tree:

Like insertions, deletion of nodes in AVL Tree is also similar to deletion in BST. After deletion of nodes, we have to check for imbalance and correct it to rebalance the tree.

Graph

A graph is a data structure that describes a binary relation between elements and has a webby looking graphical representation. Graph plays a significant role in solving a rich class of problems.

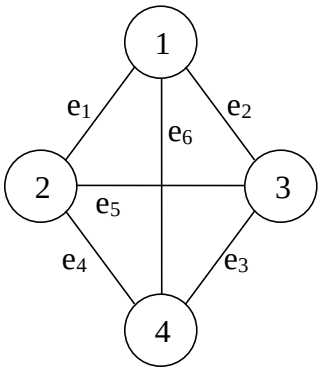
Definition:

A graph G = (V, E) consists of a finite set of non-empty set of vertices V and set of edges E.

$V = \{ v_1, v_2, \ldots, v_n\}$
$E = \{e_1, e_2, \ldots e_n\}$

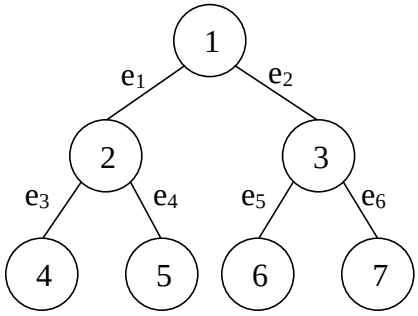
Each edge e is a pair (v, w) where v, w ϵ V. The edge is also called arc.

Eg, of graphs:

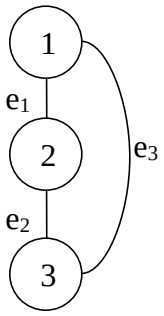


G_1

$V(G_1) = \{1, 2, 3, 4\}$
$E(G_1) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$
$e_1 = (1, 2)$
$e_2 = (1, 3)$
$e_3 = (3, 4)$
$e_4 = (2, 4)$
$e_5 = (2, 3)$
$e_6 = (1, 4)$



G_2



G_3

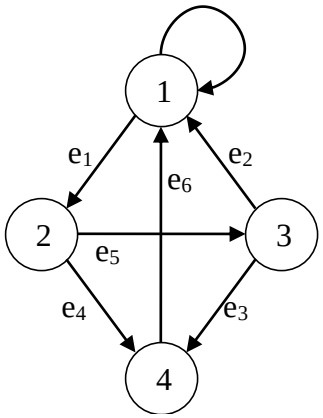
$V(G_3) = \{1, 2, 3\}$
$E(G_3) = \{e_1, e_2, e_3\}$
$e_1 = (1, 2)$
$e_2 = (2, 3)$
$e_3 = (1, 3)$

Now, specify the vertices and edges for G_2.

The vertices are represented by points or circles and the edges are line segments connecting the vertices. If the graph is directed, then the line segments have arrow heads indicating the direction.

Directed Graphs:

If every edge (i, j), in E(G) of a graph G is marked by a direction from i to j, then the graph is called directed graph. It is often called diagraph.



G_4

In edge e = (i, j), we say, e leaves i and enters j. In diagraphs, self loops are allowed. The indegree of a vertex v is the number of edge entering v. The outdegree of a vertex v is the number of edges leaving v. The sum of the indegrees of all the vertices in a graph equals to the sum of outdegree of all the vertices.

$$\sum_{i=0}^{n} d_{in}(v_i) = \sum_{i=0}^{n} d_{out}(v_i) \quad \text{n = no. of vertices}$$

Undirected Graph:

If the directions are not marked for any edge, the graph is called undirected graph. The graphs G_1, G_2, G_3 are undirected graphs. In an undirected graph, we say that an edge e = (u, v) is incident on u and v (u and v are connected).

Undirected graph don't have self loops. Incidence is a symmetric relation i.e. if e = (u, v)

Then u is a neighbor of v and vice versa. The degree of a vertex, d(v), is the total number of edges incident on it.

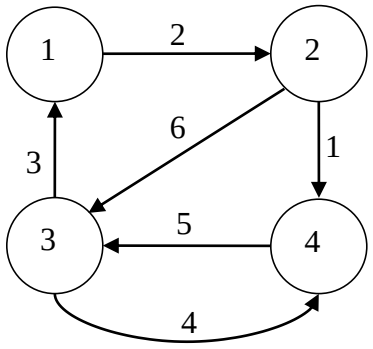
$$\sum_{i=0}^{n} d(v_i) = 2|E|$$

The sum of the degrees of all the vertices in a graph equals twice the number of edges if d(v) = 0, v is isolated. If d(v) = 1, v is pendent.

Weighted Graphs:

A graph is said to be weighted graph if every edge in the graph is assigned some weight or value. The weight is a positive value that may represent the cost of moving along the edge, distance between the vertices etc.

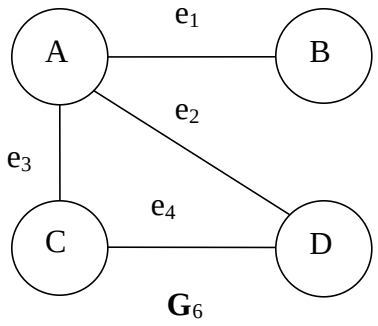
The two vertices with no edge (path) between them can be thought of having an edge (path) with weight infinite.



Additional Terminology in Graph

Adjacent and Incident

Two vertices i and j are called adjacent if there is an edge between the two. Eg. The vertices adjacent to vertex A, in the fig below are B, C, D. The adjacent vertices of C are A and D.



G$_6$

If e(i, j) is an edge on E(G), then we say that the edge e(i, j) is incident on vertices i and j. Eg. in the above figure, e_4 is incident on C and D. If (i, j) is directed edge, then i is adjacent to j and j is adjacent from i.

Path:

A path is a sequence or distinct vertices, each adjacent to the next. For e.g. the sequence of vertices e_1, e_3, e_4 (i.e (B,A), (A,C), (C,D)) of the above graph form a path from B to D. The length of the path is the number of edges in the path. So, the path from B to D has length equal to three.

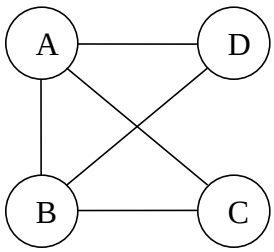
In a weighted graph, the cost of a path is the sum of the costs of its edges. Loops have path length of 1.

Cycle:

A cycle is a path containing at least three vertices such that the last vertex on the path is adjacent to the first. In the above graph G_6, A, C, D, A is a cycle.

Connected:

Any graph is *connected* provided there exists a path (directed or undirected) between any two nodes.



Connected

Complete graph

A *complete graph* is a graph in which there is an edge between every pair of vertices.

Trees and Graph

A tree is a connected graph with no cycle. A tree has **|E| = |V| - 1** edges. Since, it is connected, there is a path between any two vertices.

Representation of Graphs

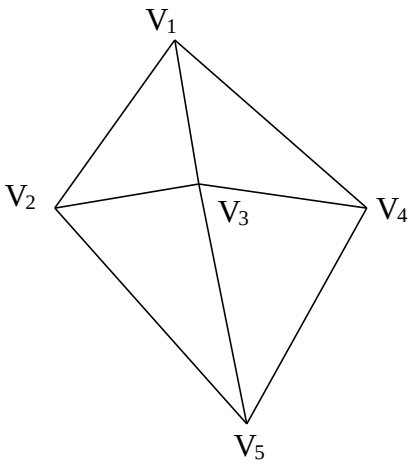
A graph can be represented in many ways. Some of them are described in this section.

Adjancency matrix

Adjancency matrix A for a graph G = (V, E) with n vertices, is n x n matrix, such that
$A_{ij} = 1$, if there is an edge from v_i to v_j
$A_{ij} = 0$, if there is no such edge

We can also write,

$$A(i, j) = \begin{cases} 1 \text{ if and only if } (v_i, v_j) \text{ exists} \\ 0 \text{ otherwise} \end{cases}$$

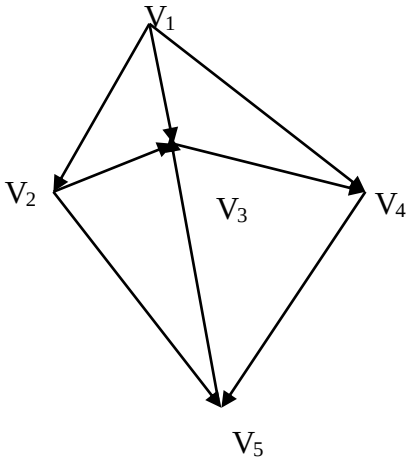


Eg; An adjacency matrix for the following undirected graph is

A =

	V1	**V2**	**V3**	**V4**	**V5**
V1	0	1	1	1	0
V2	1	0	1	0	1
V3	1	1	0	1	1
V4	1	0	1	0	1
V5	0	1	1	1	0

If the graph is directed, then the adjacency matrix will be as follows. The number in each row tells the outdegree of that vertex.



A =

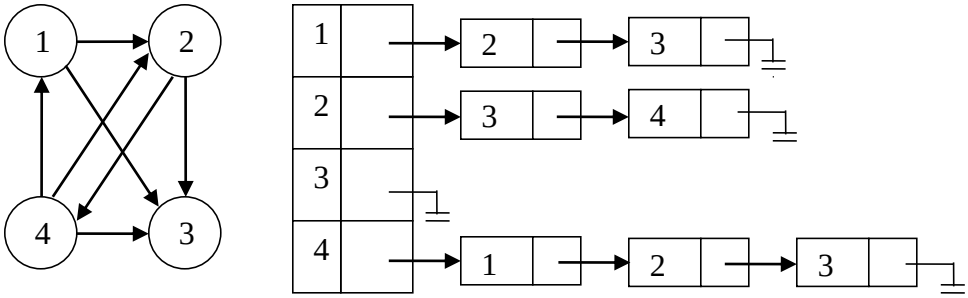
	V1	**V2**	**V3**	**V4**	**V5**
V1	0	1	1	1	0
V2	0	0	1	0	1
V3	0	0	0	1	0
V4	0	0	0	0	1
V5	0	0	1	0	0

In this representation we require n^2 bits to represent a graph with n nodes. It is a simple way to represent a graph, but it has following disadvantages:

- it takes $O(n^2)$ space
- it takes $O(n^2)$ time to solve most of the problems.

Adjacency List Representation

The n rows of an adjacency matrix can be represented as n linked lists. This is one list for each node in a graph. Each list will contain adjacent nodes. Each node has two fields, *Vertex* and *Link*. The Vertex field of a node p will contain the nodes that are adjacent to the node p.



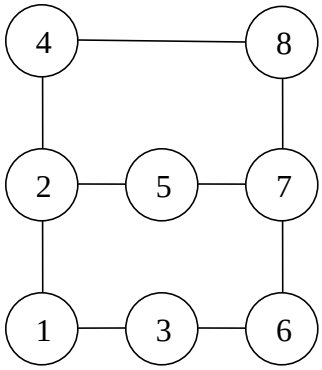
Graph Traversals

A graph traversal means visiting all the nodes of the graph. Basically, there are two methods of graph traversals.

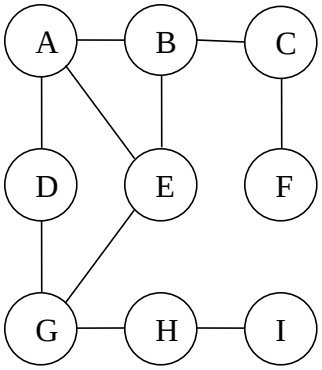
1. Breadth First Traversal
2. Depth First Traversal

1. Breadth First Traversal

The *breadth first traversal* begins with a given vertex and then it next visits all the vertices adjacent to v, putting the vertices adjacent to these in a waiting list to be traversed after all vertices adjacent to v have visited. Breadth First Traversal uses queue.



Starting from 1 Breadth First Traversal yields 1, 2, 3, 4, 5, 6, 8, 7



Starting from A Breadth First Traversal yields A, B, D, E, C, G, F, H, I

2. Depth First Traversal

The *Depth First Traversal* algorithm is roughly analogous to preorder tree traversal, the algorithm follows:

suppose that the traversal has just visited a vertex v, and let w_0, w_1, …w_k be the vertices adjacent to v

- next, visit w_0 and keep w_1, …w_k waiting
- after visiting w_0, we traverse all the vertices to which it is adjacent before returning to traverse w_1, …w_k.

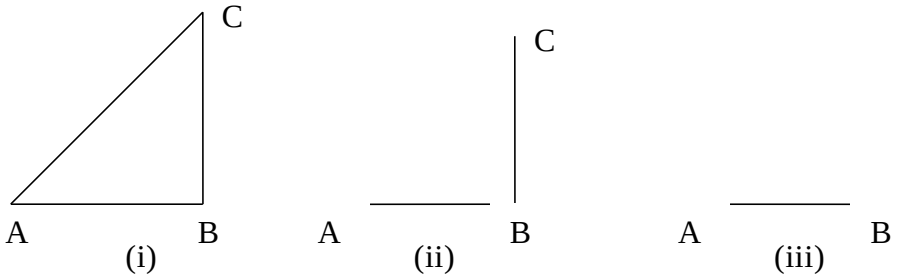
Depth first traversal uses stack to store the adjacent vertices but one.

For above 2 figures Depth First Traversal yields

	Adjacency list		**Adjacency list**
1	2, 3	A	B, D, E
2	1, 4, 5	B	A, C, E
3	1, 6	C	B, F
4	2, 8	D	A, G
5	2, 7	E	A, B, G
6	3, 7	F	C
7	5, 6, 8	G	D, E, H
8	4, 7	H	G, I
		I	H
DF Traversal:		**DF Traversal:**	
	1, 2, 4, 8, 7, 5, 6, 3		**A, B, C, F, E, G, D, H, I**

Subgraph:

Let H be a graph with vertex set v(H) and edge set E(H) and, similarly, let G be a graph with vertex set V(G) and edge set E(G). Then H is said to be a subgraph of G, written as $H \leq G$ if $V(H) \leq V(G)$ and $E(H) \leq E(G)$.

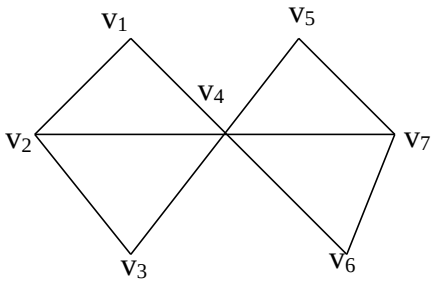


Here, second and third graph are subgraph of the first.

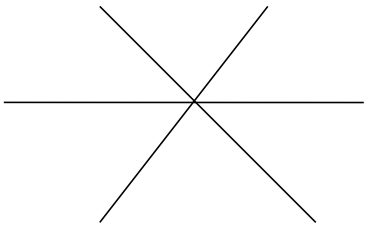
Spanning Tree:

Let a graph G=(V,E) be a graph. If T is a subgraph of G and contains all the vertices but no cycles/circuits, then 'T' is said to be spanning tree of G.

In other words, the spanning tree of an undirected graph G is the tree formed from graph edges which connects all the vertices of G.

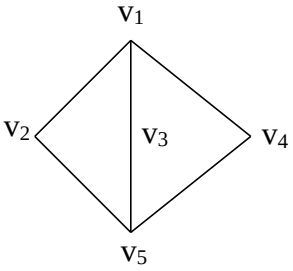


Undirected Graph G

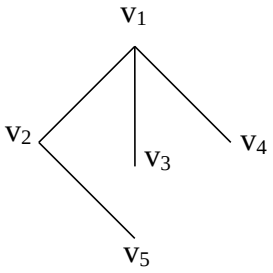


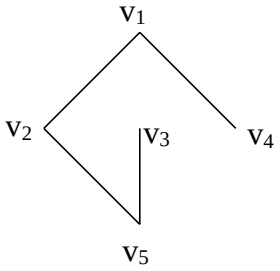
Spanning Tree of G

For a given graph G=(V, E), if Breadth-First search and Depth-First search call is made, then the resultant tree that is generated is the spanning tree of the graph G. Thus, by making a Depth-First search on G, we get the Depth-First spanning tree. Similarly, on applying Breadth-First search on graph 'G' we get Breadth First spanning tree.



Graph G





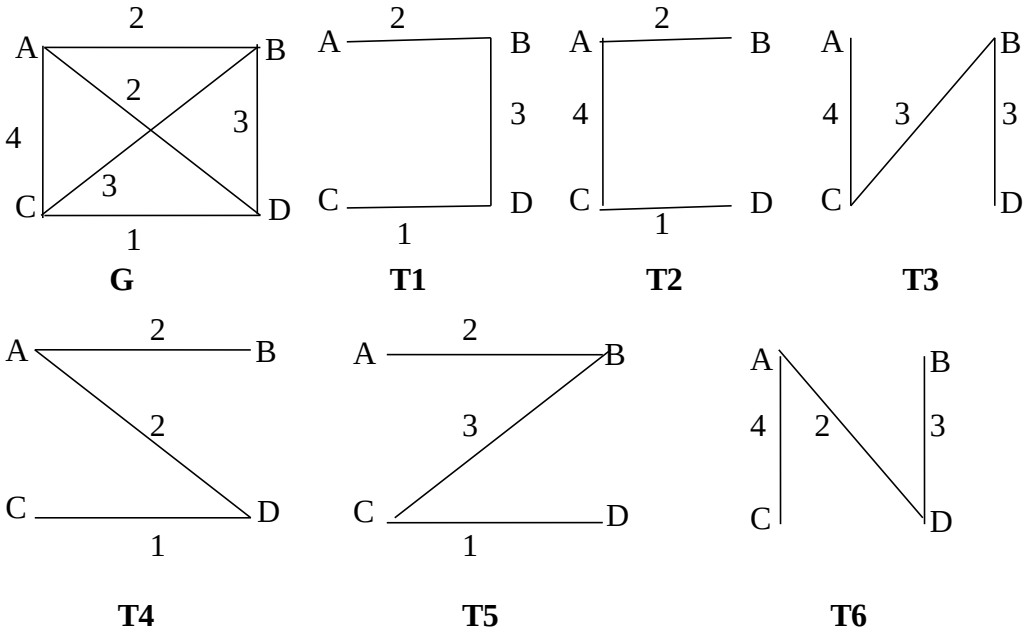
BS Spanning Tree and DF Spanning Tree of G

Minimum Spanning Tree

The cost of a spanning tree of a weighted, undirected graph is the sum of the costs (weights) of the edges in tree.

The minimum spanning tree of a weighted undirected graph is the spanning tree that connects all the vertices in G at lowest cost. So, if T is the minimum spanning tree of graph G and T' is any other spanning tree of G then,

$$W(T) \leq W(T')$$



Here, $\mathbf{T_1}$ to $\mathbf{T_6}$ are the *spanning trees* of **G**. Among them, **T4** has minimum cost (i. e. 5). So, $\mathbf{T_4}$ is the minimum spanning tree of graph G

There are several algorithms available to determine the *minimal spanning tree* of a given weighted graph.